

UNIVERSIDAD DE MÁLAGA

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN



UNIVERSIDAD DE MÁLAGA

TRABAJO FIN DE GRADO

DESARROLLO DE APLICACIONES/SISTEMAS
CON MICROBOTS BASADOS EN TIVA

GRADO EN INGENIERÍA DE
SISTEMAS ELECTRÓNICOS

Luis Miguel Campos García
MÁLAGA, 2025

Desarrollo de aplicaciones/sistemas con microbots basados en TIVA

Autor: Luis Miguel Campos García

Tutor: Ignacio Herrero Reder

Departamento: Departamento de Tecnología Electrónica

Titulación: Grado en Ingeniería de Sistemas Electrónicos

Palabras clave: TIVA, ESP32, WIFI, QT, MQTT.

Resumen

El presente Trabajo Fin de Grado tiene como objetivo la comunicación entre dos microcontroladores, Tiva TM4C123GH6PM y ESP32, así como la implementación de la funcionalidad necesaria para satisfacer el ejercicio propuesto, el cual consiste en el manejo y monitorización del movimiento del microbot, de forma inalámbrica mediante Wifi, introducido en un laberinto.

En primera instancia, dicha funcionalidad será dividida en dos modos de funcionamiento, por un lado, el modo manual, permite al usuario manejar directamente el movimiento del microbot; por otro lado, el modo automático, dicho modo cederá el control de movimiento al propio microbot.

Para el cumplimiento de lo narrado anteriormente, será hará uso y configuración de los distintos módulos necesarios para ello. El microcontrolador Tiva será el encargado de controlar los periféricos que constituyen el microbot, mientras que el microcontrolador ESP32 formará la pasarela entre la aplicación Qt y el microbot.

La aplicación Qt consistirá en un interfaz de usuario, que permite la selección del modo de funcionamiento. Con el modo manual proporcionará los controles correspondientes al tipo de movimiento (avanzar, girar, retroceder...), mientras que el modo automático aportará un gráfico del recorrido del microbot y lo que éste ve en su entorno, así como las acciones necesarias para dicho modo.

Por último, la comunicación entre dicha aplicación Qt y el microcontrolador ESP32 se realizará mediante el protocolo MQTT, donde ambas partes se suscribirán a distintos *topics*, en los cuales se publicarán la información oportuna.

Development of applications/systems with microbots based on TIVA

Author: Luis Miguel Campos García

Supervisor: Ignacio Herrero Reder

Department: Departamento de Tecnología Electrónica

Degree: Grado en Ingeniería de Sistemas Electrónicos

Keywords: TIVA, ESP32, WIFI, QT, MQTT.

Abstract

The objective of this Final Degree Project is the communication between two microcontrollers, Tiva TM4C123GH6PM and ESP32, as well as the implementation of the necessary functionality to satisfy the proposed exercise, which consists of the management and monitoring of the movement of the microbot, wirelessly via Wifi, introduced in a maze.

In the first instance, such functionality will be divided into two modes of operation, on the one hand, the manual mode, allows the user to directly manage the movement of the microbot; on the other hand, the automatic mode, such mode will yield the movement control to the microbot itself.

In order to accomplish the above, it will be necessary to use and configure the different modules required for this purpose. The Tiva microcontroller will be in charge of controlling the peripherals that make up the microbot, while the ESP32 microcontroller will form the gateway between the Qt application and the microbot.

The Qt application will consist of a user interface, which allows the selection of the operating mode. With the manual mode it will provide the controls corresponding to the type of movement (forward, turn, backward...), while the automatic mode will provide a graph of the path of the microbot and what it sees in its environment, as well as the actions required for that mode.

Finally, the communication between this Qt application and the ESP32 microcontroller will be done through the MQTT protocol, where both parts will subscribe to different topics, in which the appropriate information will be published.

Agradecimientos

Personalmente a mis abuelos, sin ellos nada de esto hubiera sido posible.
Y a toda persona que me ha aportado un pequeño empujón para conseguirlo.

*A mis padres, a mi hermana
a mí.*

Índice de la memoria

Lista de acrónimos.....	iii
Capítulo 1. Introducción.....	1
1.1 Contexto tecnológico	2
1.2 Objetivos del Trabajo Fin de Grado	5
1.3 Estructura de la memoria.....	6
Capítulo 2. Especificaciones del sistema	8
2.1 Requisitos	8
2.2 Restricciones	1
2.3 Casos de uso.....	2
2.4 Modelo de datos	3
2.5 Interfaz hardware y software	4
2.5.1 Interfaz hardware	4
2.5.2 Interfaz software	6
Capítulo 3. Desarrollo del sistema.....	8
3.1 Bloque hardware.....	9
3.1.1 Microcontroladores	9
3.1.2 Sensores	12
3.1.3 Actuadores	15
3.1.4 Fuente de alimentación.....	15
3.1.5 Chasis del microbot	16
3.1.6 Placa de expansión.....	16
3.1.7 Placa pretaladrada	17
3.2 Bloque software	18
3.2.1 Tiva C Series TM4C123GH6PM.....	18
3.2.2 ESP32-WROOM-32	31

3.2.3	Aplicación Qt.....	34
Capítulo 4.	Verificación y pruebas.....	37
4.1	Sistema de pruebas.....	37
4.2	Pruebas realizadas	38
Capítulo 5.	Conclusiones y trabajo futuro	47
5.1	Trabajo futuro	47
5.2	Conclusiones.....	49
Apéndice A.	Manual de instalación y uso	51
A.1	Manual de instalación.....	51
A.1.1	Programación del microcontrolador TIVA.....	51
A.1.2	Programación del microcontrolador ESP32.....	53
A.1.3	Instalación de la aplicación Qt.....	54
A.2	Manual de uso	55
Apéndice B.	Presupuesto de elaboración	59
Referencias.....		62

Lista de acrónimos

ADC	Analog-to-Digital Converter - Conversor Analógico-Digital
GPIO	General-Purpose Input/Output - Entrada/salida de propósito general
I2C	Inter-Integrated Circuit – Circuito Inter-Integrado
PWM	Pulse Width Modulator - Modulador de anchura de pulsos
UART	Universal Asynchronous Receivers/Transmitters - Receptor/transmisor asíncrono universal
MQTT	Message Queuing Telemetry Transport - Transporte de Telemetría de Cola de Mensajes

Capítulo 1. Introducción

Este capítulo habla sobre la historia de la robótica, desde los primeros autómatas hasta los robots actuales que conviven con las personas en diferentes áreas de la vida. Se explica cómo el avance de la microelectrónica y de la computación digital ayudó mucho a su desarrollo. Para esto, se presenta una línea de tiempo que incluye desde máquinas mecánicas sin componentes electrónicos, que aun así aportaron nuevas ideas y usos, hasta llegar a la robótica moderna, influida por los distintos contextos históricos y tecnológicos.

También se revisa cómo fueron evolucionando las capacidades de estas primeras máquinas. Pasaron de ser dispositivos muy básicos y dependientes del ser humano a los robots actuales, mucho más complejos y autónomos. Esto no solo impactó a la industria, sino también a otros aspectos de la vida diaria. Además, se explica cómo la robótica se benefició de su uso en entornos cotidianos, lo que llevó a crear nuevos tipos de robots con funciones más variadas.

Por último, el capítulo incluye una parte sobre los objetivos del proyecto y la manera en que está estructurado internamente.

1.1 Contexto tecnológico

Desde las épocas más remotas de la prehistoria llegan hasta la actualidad los registros de la evolución tecnológica que caracterizó a las sociedades pasadas en su objeto de prosperar en sus actividades más cotidianas. Por ello, no es complejo comprender el intrínseco anhelo del ser humano en cuanto al avance en materia tecnológica y, por ende, de la creación de máquinas que reproduzcan el comportamiento, movimiento y el esfuerzo que ellos mismos desarrollan.

Es por eso que inicialmente se crean los autómatas, máquinas que incorporaban dispositivos mecánicos mediante los cuales era posible ejercer el control automático de sus movimientos, aunque carentes de componentes electrónicos. Fueron diseñados y fabricados con la finalidad de asistir al ser humano en los trabajos que suponían un mayor esfuerzo físico, por tanto, su aplicación estuvo destinada prácticamente a las actividades industriales. Se nutrieron además de los avances en la mecanización y producción en cadena que trajo consigo la Revolución Industrial entre los siglos XVII y XIX [1], Pero no será hasta el año 1912 cuando pasen a denominarse “Robots”, tal y como son conocidos en la actualidad, gracias a la aportación de Karel Capek, autor de la obra teatral RUR (Rossum’s Universal Robots), donde se refiere por primera vez a estas máquinas como “*robota*” o “*roboti*” (*pl.*) y cuya traducción del checo refiere a la “servidumbre” o a los “trabajos forzados”.

En los años venideros, los robots gozaban de una popularidad que les permitió desligarse del concepto inicial que los vinculaba únicamente con el trabajo y las actividades de producción y pasar a formar parte de distintos sectores como el social, destacando en este campo la aportación de “Elektro”, el primer robot considerado humanoide, el cual contaba con capacidades dialécticas que le permitían simular una conversación e incluso caminar. Es por ello que este tipo de robots fue objeto de exhibición en ferias, lanzamiento y promoción de productos, películas y otros eventos de carácter comunitario [2] [3].

Asimismo, en los albores de la ciencia que posteriormente se conocerá como Robótica, se estaban configurando sus bases y atendiendo a materias

como la ingeniería mecánica, eléctrica, electrónica, biomédica y de ciencias de computación en las que poder apoyarse para la determinación de la misma. Como ya se ha puesto de manifiesto, esta ciencia parte fundamentalmente de los avances producidos en el ámbito de la tecnología, de los cuales surgirán nuevos progresos en cuanto a la capacidad del control automático de procesos, así como también en sistemas de control realimentados.

Buen ejemplo de ello fue la creación por parte del estudiante británico Bill Taylor, de la primera grúa automatizada conocida como “Gargantua” y fabricada mediante componentes empleados en juguetes; que supuso un gran paso para el sector industrial en 1937. Considerándose un punto de partida para los robots industriales de gran tamaño, que comenzaron a surgir tras el fin de la Segunda Guerra Mundial. Y que, en sus inicios fueron principalmente manipuladores teleoperados que contaban con escasas tareas lógicas, realizaban movimientos repetitivos y presentaban limitaciones en lo referido al seguimiento automático de señales generadas por servosistemas, así como a la regulación automática frente a perturbaciones que distanciaban el sistema de las condiciones deseadas [2] [4].

Simultánea también fue la aparición de los robots móviles, que motivada por el deseo de dotar de mayor capacidad de movimiento, alcance y autonomía a los robots dio lugar a la presentación, entre otros, de la “Tortuga de Walter” en 1948, capacitada para detectar obstáculos y reaccionar a ellos, de la misma forma que podía desplazarse a través de distintas superficies presentes en el entorno e incluso volver al punto de carga cuando su energía tendía a la insuficiencia [2].

En la línea de avances, fue en la década de los 60, cuando la empresa “Unimation”, propiedad de Joseph Engelberger y George David, pasará a considerarse la primera compañía robótica de la historia, pionera en esta categoría al patentar el primer robot de transferencia programable, conocido como “Unimate”, presentado en la Figura 1.1. Del cual acabarían vendiendo sus derechos a otras compañías, como General Motors, empresa pionera en el ámbito de la robótica industrial y demandante de la incorporación en sus cadenas de producción de estos novedosos robots [1].



Figura 1.1. Primer robot industrial, Unimate. Fuente: [5]

Fue a partir de entonces, cuando la robótica experimenta una potencial expansión, valiéndose de los adelantos que surgían en los diversos campos de los que se nutría; como del nacimiento del microprocesador en 1972, que permitiría un mayor control por computación. Pero no será hasta la “Era Robótica”, inserta en los 80, cuando alcance su máximo esplendor como consecuencia de las mejoras en microelectrónica, la tecnología de los circuitos integrados, el incremento de la capacidad computacional y el desarrollo de nuevos sensores y actuadores. Favoreciendo, por ende, al desarrollo de los ya experimentados robots móviles, que pasarán de guiarse mediante cableado bajo el suelo o a partir de sensores ópticos que respondían ante líneas trazadas en el suelo, hasta orientarse y detectar el entorno por medio de sensores láser, de ultrasonido o cámaras, entre otros [2]. Esta perfección en sus cualidades de movimiento y detección, junto a su capacidad de operabilidad en entornos diversos y cambiantes, han permitido que lejos de considerarse una fantasía futurista, constituyan actualmente una realidad palpable a partir de la aplicación de los mismos en distintos ámbitos como la exploración espacial, la logística, la atención médica, la agricultura y la industria manufacturera.

Es evidente, por tanto, que las mejoras y progresos que favorecen a los robots móviles, se verán reflejadas en los distintos tipos de robots que alberga esta categoría. Dotados de gran agilidad y aptitudes de movimiento, versatilidad en entornos adversos o espacios de reducidas dimensiones y navegación en

todas direcciones, entre otras, reúnen las capacidades idóneas para convertirse en prometedores candidatos que trasciendan a numerosas aplicaciones en el futuro, aun careciendo de popularidad en la actualidad.

Por último, y para finalizar con el recorrido que desemboca en la ya constituida ciencia de la robótica, cabe hacer mención al Internet de las Cosas (IoT). Que ha supuesto en el siglo XXI un nuevo impulso en el ámbito robótico al dotar a los robots de una interconexión a través de Internet, permitiéndoles recopilar, intercambiar y procesar datos en tiempo real. En el contexto de la robótica móvil, el IoT actúa como un facilitador clave que permite a los robots conectarse con su entorno, otros dispositivos y plataformas en la nube, mejorando su funcionalidad, autonomía y capacidad de adaptación [6].

La conectividad de alta velocidad y la creciente capacidad de procesamiento hacen que la robótica se encuentre en una posición privilegiada para aprovechar estos avances y crear sistemas cada vez más autónomos y capaces de tomar decisiones encaminadas en entornos complejos y dinámicos.

En conclusión, constituye la robótica una ciencia receptiva en constante crecimiento y desarrollo que, a pesar de haber conseguido instaurar numerosas creaciones en un amplio espectro de industrias y sectores; desde los primeros autómatas hasta los robots inteligentes y autónomos de la actualidad, no cesa en su empeño por prosperar e innovar a partir de la innata creatividad y ambición del ser humano.

1.2 Objetivos del Trabajo Fin de Grado

El objetivo principal que alberga este proyecto consiste en el diseño e implementación de una aplicación o sistema basado en el microcontrolador TIVA, que posibilite la resolución de una estructura laberíntica por parte del microbot diseñado al efecto, así como también la supervisión del proceso resolutivo desde una aplicación de escritorio.

La consecución del objetivo principal se materializa a partir de los siguientes objetivos específicos:

- Desarrollo de un algoritmo de navegación que permita al microbot identificar/localizar la salida del laberinto.

- Creación de una interfaz de usuario, mediante la cual el control y la monitorización del microbot se puedan realizar en tiempo real.
- Implementación de una comunicación inalámbrica entre el microbot y la aplicación de escritorio.
- Uso de un sensor brújula que optimice la calidad de los movimientos del microbot.
- Establecimiento de una comunicación serial con objeto de propiciar la interacción entre el microcontrolador TIVA, la brújula (I2C) y el microcontrolador (UART).

1.3 Estructura de la memoria

La memoria del presente proyecto se articula en los siguientes capítulos:

- Introducción:
El apartado introductorio aborda la evolución de la robótica a lo largo del tiempo, así como los principales avances que propician su desarrollo hasta llegar a la robótica contemporánea. Asimismo, incluye la descripción de los objetivos previstos para el presente proyecto. Y concluye con la estructuración de la memoria y la definición de cada una de sus partes.
- Especificaciones del sistema:
Esta sección engloba las especificaciones del sistema, así como la descripción de las funciones a conseguir, los requisitos que habrán de cumplirse, las restricciones impuestas y los posibles casos de uso.
Se incluye, además, un inventario descriptivo de los usuarios del sistema, modelos de datos, interfaces hardware y software.
- Desarrollo del sistema:
Se desarrolla en este punto la descripción detallada de la implementación del sistema a partir de la diferenciación entre la parte hardware del sistema, que presenta y justifica los componentes empleados; y la parte software del sistema, que incluye las rutinas y funciones implementadas que permiten el correcto funcionamiento del sistema.
- Verificación y pruebas:

Quedan recogidas en esta sección las pruebas de verificación sobre el sistema que han sido realizadas con el fin de comprobar el correcto funcionamiento de éste.

- Manual de instalación y uso:

El manual de instalación y uso recoge las indicaciones necesarias para la puesta en funcionamiento del sistema y describe su utilización.

- Conclusiones y trabajo futuro:

En esta última sección del trabajo final de grado se comentan las experiencias y conocimientos adquiridos mediante el desarrollo del sistema. Asimismo, contiene una serie de propuestas encaminadas a la mejora del sistema o las posibles extensiones del mismo en futuros proyectos.

- Apéndice A:

Este apéndice agrupa los pasos a seguir para la programación de los microcontroladores usados en el proyecto, así como un guía de uso de la aplicación Qt desarrollada.

- Apéndice B:

En este apéndice se calcula el coste económico del proyecto realizado.

- Referencias:

En este apartado se proporciona un listado con las referencias objeto de consulta en la elaboración del sistema.

Capítulo 2. Especificaciones del sistema

El sistema desarrollado en este TFG se encargará de controlar los motores que mueven el microbot, procesar la información de sus sensores y actuar en consecuencia. También debe poder comunicarse con los microcontroladores involucrados y con la aplicación que controla y supervisa el microbot. Para lograrlo, es necesario cumplir con los requisitos mencionados en el siguiente apartado y tener en cuenta ciertas restricciones establecidas desde el inicio.

Además, este capítulo presenta los casos de uso y los usuarios del sistema, así como una descripción de los modelos de datos y de las interfaces de hardware y software que se van a utilizar.

2.1 Requisitos

Consisten los requisitos en condiciones o circunstancias que se establecen con motivo de la determinación del funcionamiento y capacidades del sistema a desarrollar.

En este caso, han sido establecidos un total de catorce requisitos con diversas motivaciones, quedan reflejados en la tabla 2.1., a continuación. En ella se detalla el nombre del requisito (por ejemplo: “Requisito Brújula”), una descripción del mismo (por ejemplo: “Debe poder realizar giros de 90°”), así como la tipología de los requisitos, y, por último, la prioridad que se concede a cada uno.

Id.	Nombre	Descripción	Tipo	Prioridad
1	Requisito Sistema	Implementar un sistema capaz de resolver un laberinto basado en TIVA.	Funcional	1
1.1	Requisito Microbot	Debe poder desplazarse por el laberinto y comunicarse con el cliente MQTT.	Funcional	1
1.1.1	Requisito Movimiento	Debe poder desplazarse en todas direcciones y realizar giros.	Funcional	1
1.1.1.1	Requisito Motor	Debe poder controlar las ruedas del microbot.	Funcional	1
1.1.1.2	Requisito Brújula	Debe poder realizar giros de 90 grados.	Funcional	1
1.1.2	Requisito Sensores Distancia	Debe poder detectar paredes en caso de haberlas.	Funcional	1
1.1.3	Requisito Comunicación	Debe poder comunicarse con el cliente MQTT.	Funcional	1
1.1.3.1	Requisito Comunicación ESP32	El microcontrolador TIVA debe ser capaz de comunicarse con el microcontrolador ESP32.	Funcional	1
1.1.3.2	Requisito Comunicación Wifi	El microcontrolador ESP32 debe poder publicar en el cliente MQTT.	Funcional	1
1.1.4	Requisito Finalización	Debe poder finalizar el movimiento al completar el laberinto.	Funcional	2
1.2	Requisito Aplicación	Debe poder controlar y monitorizar el movimiento del microbot.	Funcional	1
1.2.1	Requisito MQTT	Debe poder recibir y enviar información.	Funcional	1
1.2.1.1	Requisito Control	La aplicación debe poder controlar el microbot.	Funcional	1
1.2.1.2	Requisito Monitorización	La aplicación debe poder monitorizar el movimiento del microbot.	Funcional	2

Tabla 2.1. Tabla de descripción de requisito

De la misma forma, estos requisitos han sido organizados en función de su jerarquía mediante un diagrama, expuesto a continuación en la Figura 2.1., con objeto de facilitar su interpretación.

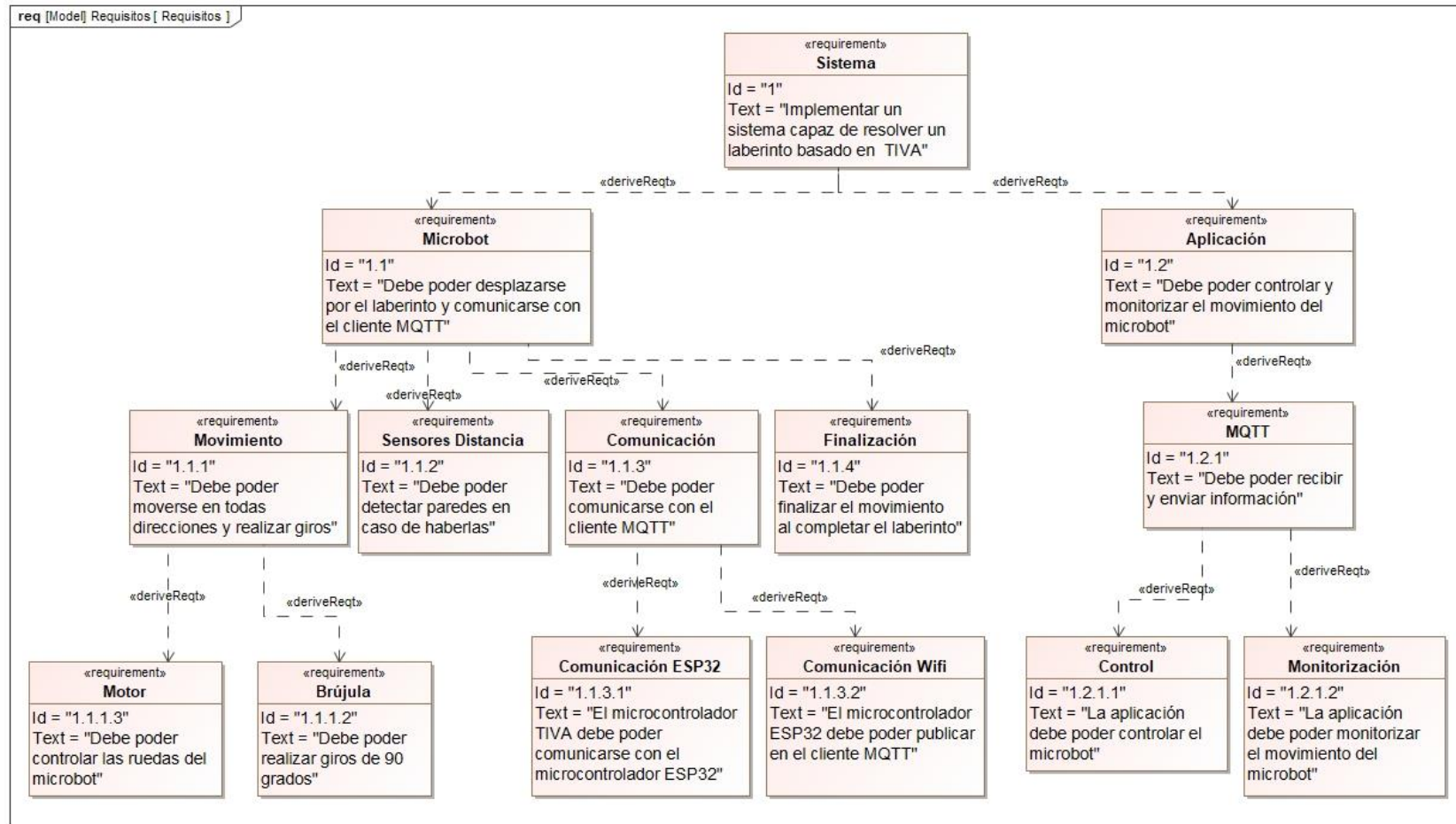


Figura 2.1. Diagrama de requisitos del sistema

2.2 Restricciones

En este apartado se describen las restricciones impuestas para el desarrollo del sistema, primeramente, a partir de una tabla que reúne datos relativos a la denominación de cada limitación, una breve descripción de la misma y, finalmente, el bloque al que afecta directamente.

Id.	Nombre	Descripción	Bloque Afectado
1	Uso de TIVA	Debe hacerse uso del microcontrolador TIVA.	Microbot
2	Uso de protocolo de comunicación serie	Debe usarse al menos un protocolo de comunicación serie.	UART, I2C
3	Uso de protocolo de comunicación inalámbrica	Debe usarse un protocolo de comunicación inalámbrica.	Módulo Wifi
4	Uso de nuevos sensores/actuadores	Debe usarse un sensor o actuador no utilizado previamente en la asignatura Microbótica.	Sensor brújula

Tabla 2.2. Tabla de descripciones de restricciones

Finalmente, se plasma la información recogida en la Tabla 2.2, en un diagrama, Figura 2.2, creado con objeto de proporcionar una interpretación más visual en cuanto a las restricciones y los bloques afectados.

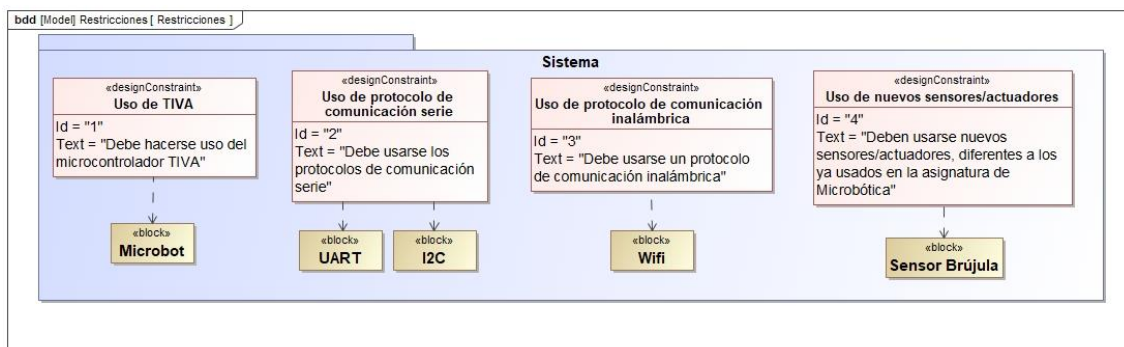


Figura 2.2. Diagrama de restricciones del sistema

2.3 Casos de uso

En este apartado se describe como el usuario interactúa con el sistema, mostrando las relaciones entre dicho usuario y las funciones o servicios que el sistema proporciona.

El usuario es el encargado principal del funcionamiento del sistema. En primera instancia, debe encender el microbot, una vez encendido se establece una conexión con una red wifi, gracias al uso del módulo wifi integrado en el microcontrolador ESP32. Una vez conectado a la red wifi procede a la conexión con un cliente MQTT, el cual permite la suscripción a los distintos *topics* que propician el envío y recepción de información entre el microbot y la aplicación Qt. A continuación, maneja la aplicación Qt, la cual se conecta al mismo cliente y se suscribe a los *topics* oportunos para establecer la conexión con el microbot, que habilita la selección del modo de funcionamiento del microbot:

- Modo manual: el usuario controla el movimiento del microbot mediante los botones de desplazamiento de la interfaz.
- Modo automático: el microbot es el encargado de controlar su propio movimiento, dicho modo es el destinado para la resolución del laberinto, poniendo en marcha el algoritmo de navegación implementado. A medida que el microbot avanza una distancia preestablecida, analiza el entorno y envía la información de éste hacia la aplicación donde se representa su recorrido y el entorno detectado.

En caso de que el usuario seleccione el modo automático, debe colocar el robot en el inicio del laberinto. Por último, en el supuesto de que el microbot complete el laberinto, detiene su movimiento y finaliza su funcionamiento.

Para obtener una visión más concreta de los casos de uso y las relaciones comentadas anteriormente, se puede observar la Figura 2.3.

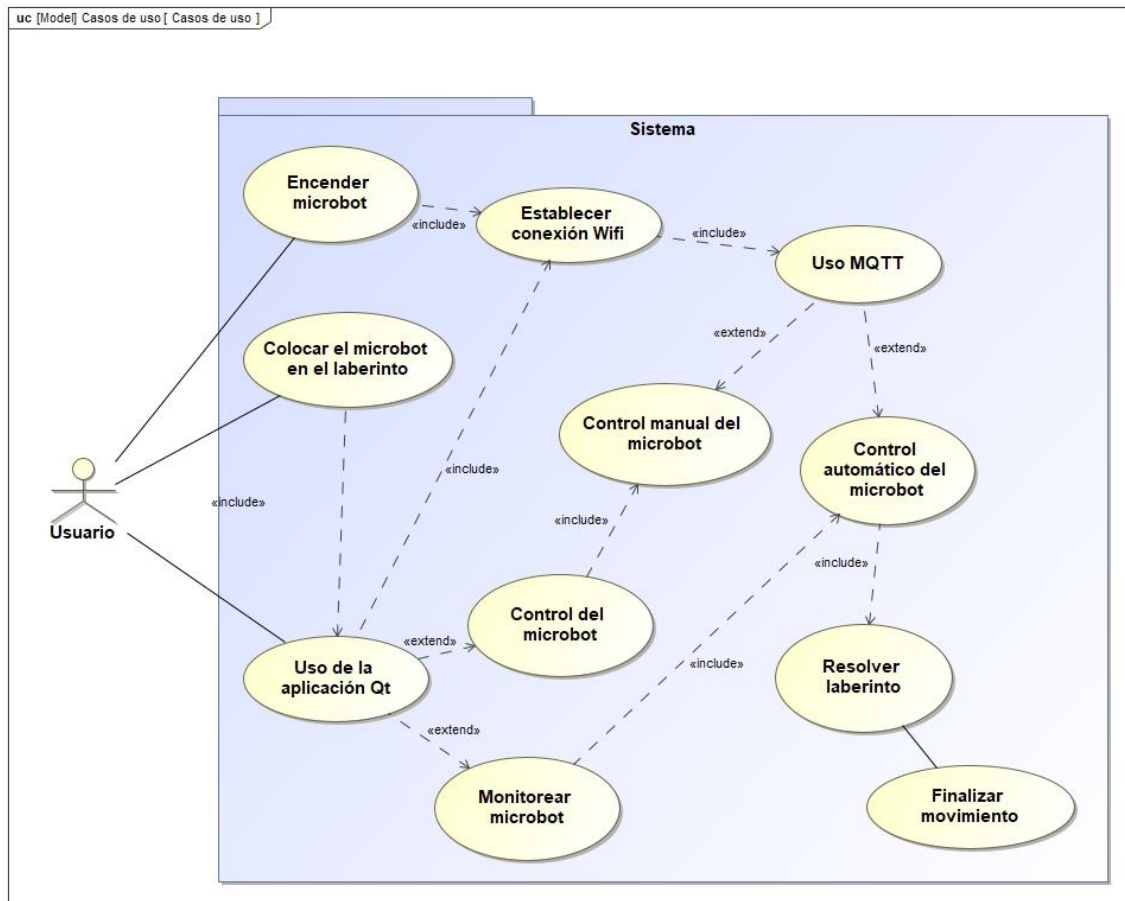


Figura 2.3. Diagrama de los casos de uso del sistema

2.4 Modelo de datos

En este apartado se describe e indica como se estructuran los modelos de datos utilizados en el sistema. Para un mejor entendimiento, se desarrolla un diagrama para el caso del microbot, mostrado en la Figura 2.4, y otro para el caso de la aplicación Qt, mostrado en la Figura 2.5, donde se dividen por clases y se realiza una asociación y composición de ellas.

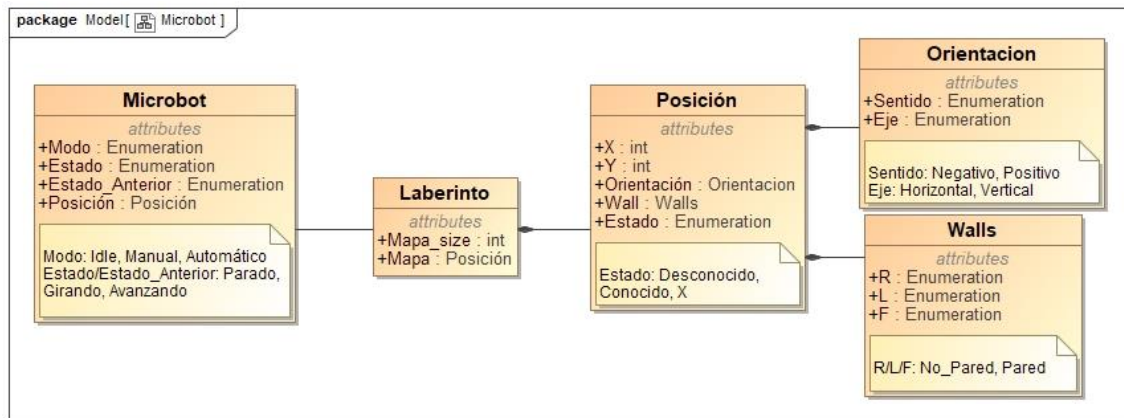


Figura 2.4. Diagrama del modelo de datos del microbot

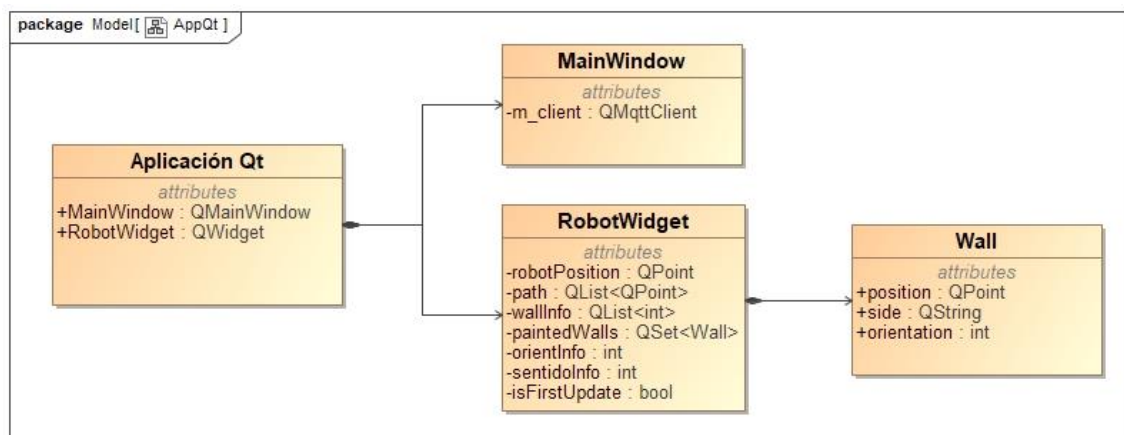


Figura 2.5. Diagrama del modelo de datos de la aplicación Qt

2.5 Interfaz hardware y software

En este apartado se realiza un breve listado de los componentes que configuran los interfaces empleados para el desarrollo del sistema, siendo éstos explicados con más detalle en el próximo capítulo.

2.5.1 Interfaz hardware

Los componentes hardware utilizados son los que sustancialmente componen el microbot:

- Microcontrolador TIVA C Series TM4C123G:
 - Módulo UART
 - Módulo I2C
 - Módulo ADC
 - Módulo PWM

- Puertos GPIO
- Microcontrolador ESP32:
 - Módulo UART
 - Módulo Wifi
 - Puerto GPIO
- Actuadores:
 - Motor derecho
 - Motor izquierdo
- Sensores:
 - Sensores de distancia
 - Sensor encoder
 - Sensor brújula
 - Sensor suelo
- Placa de expansión
- Placa pretaladrada
- Chasis del microbot
- Fuente de alimentación

Dicho listado de componentes ha sido recogido en un diagrama, Figura 2.6, para una mejor interpretación visual.

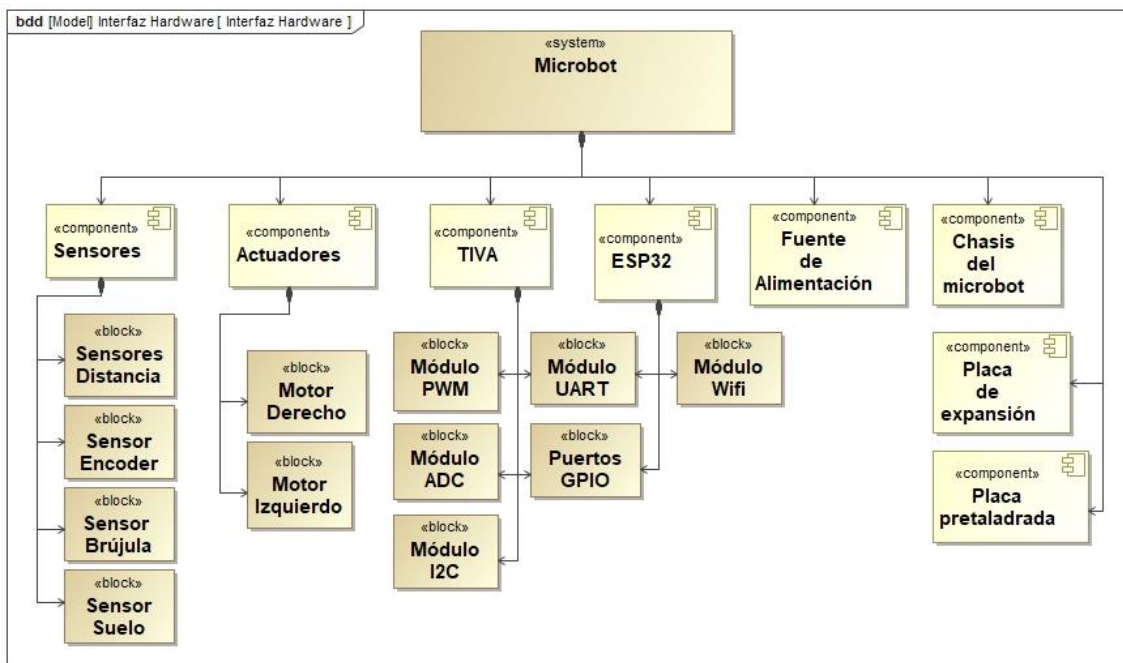


Figura 2.6. Diagrama de la interfaz hardware del sistema

2.5.2 Interfaz software

Este apartado concierne a los drivers empleados para el control de los componentes hardware, la gestión lógica del sistema y proveer de servicios al resto del sistema.

En la parte correspondiente al microbot:

- Microcontrolador TIVA C Series TM4C123G:
 - *Motor.h/Motor.c*
 - *ComUART.h/ComUART.c*
 - *SenDistancia.h/SenDistancia.c*
 - *ComI2C.h/ComI2C.c*
 - *Brujula.h/Brujula.c*
 - *Navegacion.h/Navegacion.c*
 - *Tiva_commons.h*
 - *tm4c123gh6pm_startup_ccs.c*
 - *main.c*
 - Drivers propios del microcontrolador
 - *FreeRTOS.h*
 - *semphr.h*
 - *task.h*
 - *queue.h*
- Microcontrolador ESP32:
 - Drivers propios del microcontrolador
 - *FreeRTOS.h*
 - *semphr.h*
 - *task.h*
 - *queue.h*
 - *mqtt_client.h*
 - *nvs_flash.h*
 - *uart.h*
 - *gpio.h*
 - *app_main.c*

En cuanto a la parte propia de la aplicación Qt:

- Drivers propios de Qt
- *robotwidget.h/robotwidget.cpp*
- *mainwindow.h/mainwindow.cpp*

De igual forma, para una mayor comprensión se ha recogido el anterior listado es un diagrama, Figura 2.7, donde se puede apreciar la integración de drivers propios de los fabricantes (TIVAware, Expressif, Qt), de terceros (FreeRTOS) y los propiamente implementados por el autor.

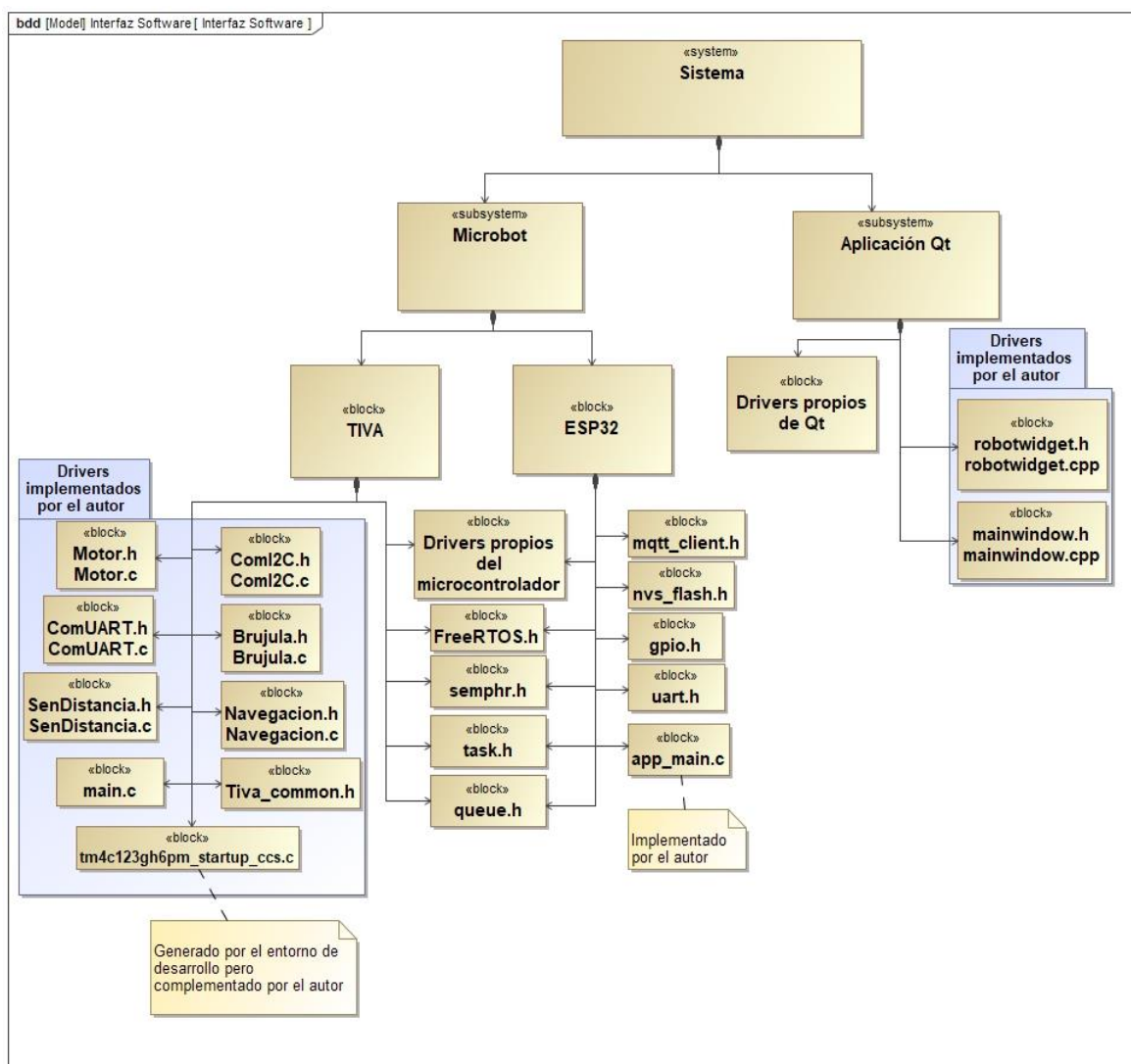


Figura 2.7. Diagrama de la interfaz software del sistema

Capítulo 3. Desarrollo del sistema

Como se ha comentado en apartados anteriores, el presente capítulo constituye una descripción y explicación más detallada de las partes implicadas en el desarrollo del sistema. Para ello, este capítulo es dividido en dos bloques, uno referente a la parte hardware y otro a la parte software, pero no sin antes mencionar el funcionamiento del sistema.

El ejercicio previsto para el sistema consiste en una prueba ya establecida en la asignatura de Microbótica, la cual radica en la resolución de un laberinto por parte del microbot, haciendo uso de los sensores y actuadores que se consideren oportunos para su culminación, así como de la lógica necesaria para la configuración y control de la parte hardware implicada, de igual forma, se hace hincapié en la implementación de algún algoritmo que satisfaga dicho ejercicio.

El funcionamiento se basa en el uso de una aplicación Qt, siendo ésta la única forma de interactuar con el microbot, sin contar con su encendido y colocación en el laberinto, al ser ambos clientes MQTT. Mediante la aplicación es posible la selección del modo de funcionamiento del microbot, el cual puede ser manual o automático.

En cuanto al modo manual, la aplicación proporciona al interfaz de usuario los controles direccionales necesarios para el movimiento del microbot. Por otro lado, el modo automático, que es el destinado para la resolución del laberinto,

cede tanto el control como la determinación del movimiento al microbot, basándose en el algoritmo implementado.

3.1 Bloque hardware

3.1.1 Microcontroladores

Con el afán de satisfacer los requisitos establecidos, es necesario el uso de un microcontrolador adicional, ya que el microcontrolador TIVA carece de módulo wifi, por tanto, se hace uso del microcontrolador ESP32 para sufragar esta necesidad.

3.1.1.1 Tiva C Series TM4C123GH6PM

Este microcontrolador, perteneciente al fabricante Texas Instruments, es el encargado de controlar los distintos componentes hardware que conforman el microbot, haciendo uso de los módulos pertinentes para ello.

A continuación, en la Figura 3.1, se muestran los parámetros, y entre ellos, los módulos que forman este microcontrolador.

Frequency (MHz)	80
Flash memory (kByte)	256
RAM (kByte)	32
ADC type	12-bit SAR
Number of GPIOs	43
UART	8
Number of I2Cs	4
Features	2 Analog / 16 Digital Comparators, 2 QEI Channels, Hibernation module
Operating temperature range (°C)	-40 to 105
Ethernet	No
PWM (Ch)	16
SPI	4
USB	Yes
CAN (#)	2
Rating	Catalog
Communication interface	CAN, I2C, SSI/SPI, UART, USB 2.0 OTG/host/device

Figura 3.1. Parámetros del microcontrolador TM4C123GH6PM Fuente: [7]

- Módulo UART: encargado de la comunicación serie con el microcontrolador ESP32, permitiendo la recepción de comandos por parte la aplicación Qt y envío de la información recogida del entorno por el microbot a ésta. Se hace uso del módulo UART2, donde la recepción se hace a través del pin PD6 y la transmisión por el pin PD7.
- Módulo I2C: encargado de comunicar el microcontrolador con el sensor brújula. Se emplea el módulo I2C1, en el cual se utiliza el pin PA6 para la señal de reloj y el pin PA7 para la señal de datos.
- Puertos GPIO: encargados de obtener información del Encoder y el sensor del suelo conectados a los pines PB4 y PB6, respectivamente.

3.1.1.2 ESP32-WROOM-32

Este microcontrolador, fabricado por Espressif, es el encargado de crear la pasarela entre el microbot y la aplicación Qt, de igual forma que con el microcontrolador TIVA, se presentan las especificaciones de éste, Figura 3.3, y el *pinout* de sus módulos, Figura 3.4.

Categories	Items	Specifications
Certification	RF certification	See certificates for ESP32-WROOM-32
	Wi-Fi certification	Wi-Fi Alliance
	Bluetooth certification	BQB
	Green certification	RoHS/REACH
Test	Reliability	HTOL/HTSL/uHAST/TCT/ESD
Wi-Fi	Protocols	802.11 b/g/n (802.11n up to 150 Mbps) A-MPDU and A-MSDU aggregation and 0.4 μ s guard interval support
	Center frequency range of operating channel	2412 ~ 2484 MHz
Bluetooth	Protocols	Bluetooth v4.2 BR/EDR and Bluetooth LE specification
	Radio	NZIF receiver with -97 dBm sensitivity
		Class-1, class-2 and class-3 transmitter
	Audio	AFH CVSD and SBC
Hardware	Module interfaces	SD card, UART, SPI, SDIO, I2C, LED PWM, Motor PWM, I2S, IR, pulse counter, GPIO, capacitive touch sensor, ADC, DAC, Two-Wire Automotive Interface (TWAI®), compatible with ISO11898-1 (CAN Specification 2.0)
	Integrated crystal	40 MHz crystal
	Integrated SPI flash ¹	4 MB
	Operating voltage/Power supply	3.0 V ~ 3.6 V
	Operating current	Average: 80 mA
	Minimum current delivered by power supply	500 mA
	Recommended operating ambient temperature range	-40 °C ~ +85 °C
	Package size	18 mm × 25.5 mm × 3.10 mm

Figura 3.3. Especificaciones del microcontrolador ESP32-WROOM-32 Fuente: [9]

Dichos sensores hacen uso de los módulos ADC0 y ADC1, y se han conectado al microcontrolador TIVA de la siguiente forma:

- Sensor de distancia frontal: conectado al pin PE3, correspondiente a la entrada analógica número 0 (AIN0).
- Sensor de distancia izquierdo: conectado al pin PE2, correspondiente a la entrada analógica número 1 (AIN1).
- Sensor de distancia derecho: conectado al pin PE1, correspondiente a la entrada analógica número 2 (AIN2).

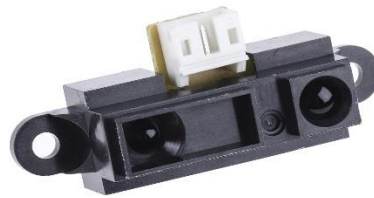


Figura 3.5. Sensor GP2Y0A41SK0F [11]

3.1.2.2 Sensor Encoder

Este sensor es el encargado de activar el reconocimiento del entorno del microbot transcurrida cierta distancia en el laberinto, por lo que se hace uso de él únicamente en el modo automático, hace uso del puerto GPIOB y está conectado al microcontrolador TIVA a través del pin PB4.



Figura 3.6. Sensor CNY70 [12]

3.1.2.3 Sensor brújula

Este sensor puede ser utilizado en ambos modos de funcionamiento y se encarga de asistir al microbot para realizar giros de 90 grados. Hace uso del módulo I2C1 del microcontrolador TIVA mediante los pines PA6 (señal de reloj, SCL) y PA7 (señal de datos, SDA). Dicho sensor ya consta de resistencia de pull-up internas en sus líneas, por lo que no son necesarias añadirlas externamente.



Figura 3.7. Sensor GY-271 [13]

3.1.2.4 Sensor del suelo

Este sensor se encarga de detectar el final del laberinto, caracterizado por un suelo de tonalidad oscura (principalmente negro), es usado únicamente en el modo automático de funcionamiento y permite concluir el algoritmo de navegación del microbot. Dicho sensor hace uso del puerto GPIOB y se conecta al microcontrolador TIVA a través del pin PB6.



Figura 3.8. Sensor TCRT1000 [12]

3.1.3 Actuadores

Los actuadores de los consta el microbot son dos servo-motores encargados del movimiento de éste, son usados en cualquier modo de funcionamiento siempre que se quiera desplazar el microbot. Hacen uso del módulo PWM1 y se conectan al microcontrolador TIVA de la siguiente forma:

- Motor derecho: conectado al pin PF2 correspondiente con la salida del módulo PWM1 número 6 (M1PWM6).
- Motor izquierdo: conectado al pin PF3 correspondiente con la salida del módulo PWM1 número 7 (M1PWM7).



Figura 3.9. Servomotor Futaba S3003 [14]

3.1.4 Fuente de alimentación

Consiste en una batería de tamaño y amperaje apropiado para el microbot, dotándolo de independencia de cableado para su alimentación.



Figura 3.10. Fuente de alimentación

3.1.5 Chasis del microbot

El chasis del microbot consta de varias partes, la carcasa que constituye el cuerpo del microbot, las ruedas y el soporte para el sensor encoder. Está fabricado con impresión 3D y proporcionado por la escuela.

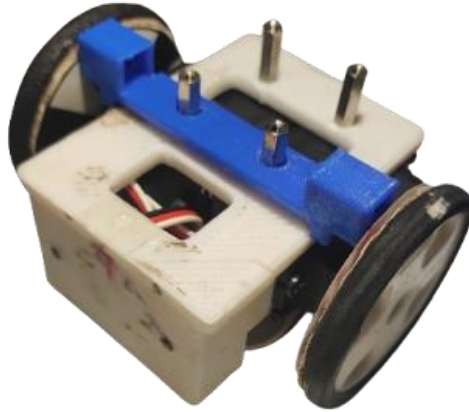


Figura 3.11. Chasis del microbot

3.1.6 Placa de expansión

La placa de expansión consiste en una PCB proporcionada por la escuela, que permite el anclaje del microcontrolador TIVA, dando un mejor acceso a los pines que controlan los motores del microbot y proporcionando más pines GND, 3,3 V y 5 V.

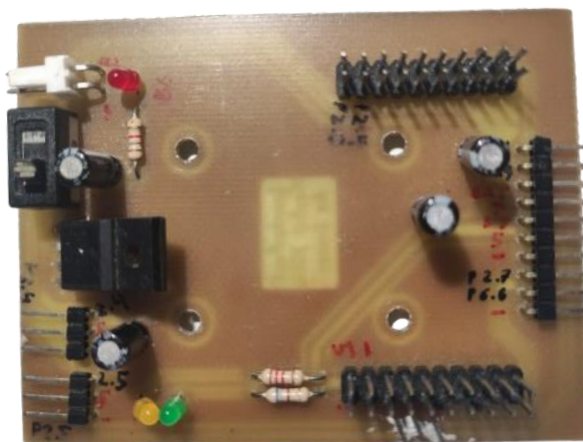


Figura 3.12. Placa de expansión

3.1.7 Placa pretaladrada

Dicha placa tiene una función similar a la placa de expansión, proporciona un anclaje para el microcontrolador ESP32, mejor acceso a los diferentes pines, usados para el control de los componentes, procedentes del microcontrolador TIVA, y un mayor número de pines GND, 3,3 V y 5 V.

Permite una alimentación directa del microcontrolador ESP32 cuando se alimenta el microcontrolador TIVA, de igual forma, conecta los módulos UART de ambos microcontroladores sin necesidad de cableado. En dicha placa se encuentra el sensor brújula, suministrándole la alimentación y GND sin necesidad de cables, véase la Figura 3.13.

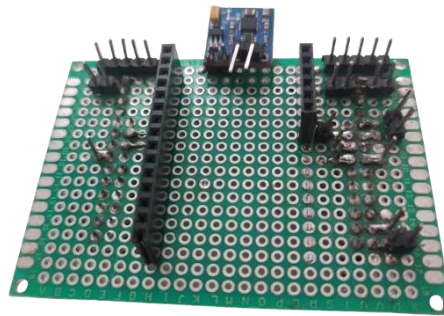


Figura 3.13. Placa pretaladrada

Para finalizar con el bloque hardware se muestra, en la Figura 3.14, el microbot montado con todos sus componentes hardware.

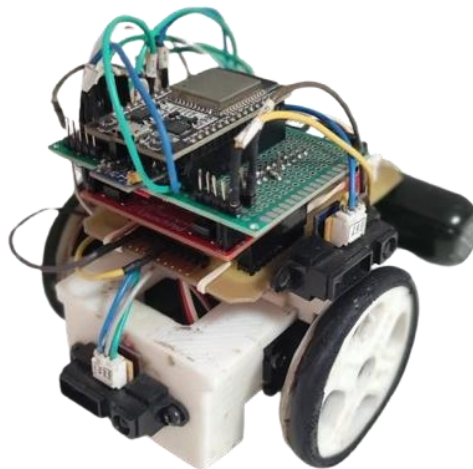


Figura 3.14. Aspecto del microbot

3.2 Bloque software

Este apartado corresponde con el bloque software del sistema, donde se describen detalladamente las funciones y estructuras de datos implementadas para la lógica de control y el algoritmo de navegación, así como para la comunicación inalámbrica y el desarrollo y funcionamiento de la aplicación Qt. De igual forma, se da explicación a las decisiones tomadas para dichas implementaciones y los valores usados en el contexto general del proyecto.

Dicho bloque se divide en tres apartados correspondientes con los núcleos donde reside el desarrollo software realizado.

3.2.1 Tiva C Series TM4C123GH6PM

En este apartado se recogen los drivers implementados para el uso de los componentes conectados al microcontrolador TIVA. Para el desarrollo de éstos, se sirve del entorno de desarrollo Code Composer Studio, proporcionado por el fabricante Texas Instruments [15].

En el proyecto, que conforman estos drivers, se hace uso del Sistema Operativo de tiempo real FreeRTOS, el cual permite la ejecución de múltiples tareas simultáneamente, una gestión eficiente de las interrupciones y recursos, entre algunas de sus características principales [16].

3.2.1.1 Driver para el uso de los motores

Driver: *Motor.c* *Motor.h*

Defines:

```

14/*****
15 * Defines
16 *****/
17 #define PERIOD_PWM          50000          // Ciclos de reloj para conseguir una señal periódica de 50Hz
18                                           // (según reloj de periférico usado)
19
20 // Rueda Derecha          (PWM_OUT_6)          // Anchos de pulso en ticks de reloj del modulo PWM
21 #define ADELANTE_DRC        3457          // para seleccionar el movimiento de la rueda derecha
22 #define ATRAS_DRC           4065          // conectada al motor.
23 #define PARADA_DRC          3800          //
24 #define ADELANTE_LENTO_DRC  3575          //
25 #define ATRAS_LENTO_DRC     3980          //
26
27 // Rueda Izquierda        (PWM_OUT_7)          // Anchos de pulso en ticks de reloj del modulo PWM
28 #define ADELANTE_IZQ        4075          // para seleccionar el movimiento de la rueda izquierda
29 #define ATRAS_IZQ           3450          // conectada al motor.
30 #define PARADA_IZQ          3830          //
31 #define ADELANTE_LENTO_IZQ  3950          //
32 #define ATRAS_LENTO_IZQ     3735          //

```

Figura 3.15. Definiciones del driver para el uso de los motores

El cálculo de la definición `PERIOD_PWM` se efectúa como se indica en la Ecuación 3.2:

$$T = \frac{1}{f} = \frac{1}{50} = 0,02 \text{ s} \quad (3.1)$$

$$Ciclos = f_{mod} * T = 2,5 * 10^6 * 0,02 = 50000 \text{ ciclos} \quad (3.2)$$

donde T es el periodo de la señal periódica de 50 Hz, f es la frecuencia de la señal periódica y f_{mod} es la frecuencia del módulo PWM establecida, que es una fracción de la frecuencia de reloj del sistema.

Para el cálculo del resto de definiciones se hace uso de la Ecuación 3.2, igualmente que, en el caso anterior, pero atendiendo a un periodo T diferente dependiendo del tipo de movimiento deseado:

- En el caso de máxima velocidad de la rueda en un sentido, se tiene un periodo $T = 1 \text{ ms}$.
- En el caso de parada, se tiene un periodo $T = 1,52 \text{ ms}$.
- En el caso de máxima velocidad de la rueda en el otro sentido, se tiene un periodo $T = 2 \text{ ms}$.

Estos valores de periodo provienen de la información proporcionada en la asignatura de Microbótica para el uso de los motores. Por último, los ciclos obtenidos como resultando han sido modificados a conciencia para adaptar el correcto funcionamiento de cada motor.

Funciones:

```

34 /*****
35 *  Funciones
36 *****/
37 void Motor_Init();           // Función para configurar el modulo PWM que controla los motores
38 void Avanzar();              // Función para hacer avanzar el robot
39 void Retroceder();           // Función para hacer retroceder el robot
40 void Parar();                // Función para parar el robot
41 void Girar_Drc();             // Función para girar el robot hacia la derecha
42 void Girar_Izq();            // Función para girar el robot hacia la izquierda
43 void GirarLento_Izq();       // Función para girar el robot hacia la derecha lentamente
44 void GirarLento_Drc();       // Función para girar el robot hacia la izquierda lentamente

```

Figura 3.16. Funciones del driver para el uso de los motores

A continuación, se explican las funciones con las que cuenta el driver, Figura 3.16:

- **Motor_Init():** se habilita tanto el puerto GPIOF, en el que residen los pines utilizados como salidas PWM, como el módulo PWM1, encargado de generar las señales, de igual forma se habilitan para su uso en modo de bajo consumo. También, se configuran los pines específicos para ser usados por el módulo PWM, y se habilitan las salidas del módulo PWM conectadas a estos pines. Luego, se selecciona el generador que utiliza el módulo y su modo de funcionamiento, que realiza la cuenta hacia abajo. Por último, se carga un estado inicial de parada para las ruedas y se habilitar el generador del módulo.
- **Avanzar():** carga los valores específicos, dependiendo del modo de funcionamiento del microbot, para un estado de avance de las ruedas. En el caso del modo automático, la velocidad es más lenta que en el modo manual.
- **Retroceder():** carga los valores específicos para un estado de retroceso de las ruedas.
- **Parar():** carga los valores específicos para un estado de parada de las ruedas.
- **Girar_Drc():** carga los valores específicos para un estado de giro hacia la derecha de las ruedas.
- **Girar_Izq():** carga los valores específicos para un estado de giro hacia la izquierda de las ruedas.
- **GiroLento_Izq():** carga los valores específicos para un estado de giro lento hacia la izquierda de las ruedas.
- **GirarLento_Drc():** carga los valores específicos para un estado de giro lento hacia la derecha de las ruedas.

Las funciones de giros lentos son las usadas cuando se desea que el microbot gire 90 grados, permitiendo que el sensor brújula obtenga mejores mediciones del ángulo.

3.2.1.2 Driver para el uso de los sensores de distancia

Driver: *SenDistancia.c* *SenDistancia.h*

Tipo de datos:

```

23 uint8_t Lista_Distancia[27] = {4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
24                               20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30};
25
26 uint16_t Lista_ADCF[27] = {3170, 2675, 2290, 2105, 1850, 1640, 1500, 1380, 1275, 1150, 1055, 985,
27                             935, 880, 835, 785, 765, 710, 690, 665, 640, 620, 595, 570, 545, 520, 520};
28
29 uint16_t Lista_ADCI[27] = {3410, 2820, 2500, 2135, 1885, 1725, 1545, 1390, 1300, 1205, 1095, 995, 965,
30                             855, 805, 755, 730, 685, 660, 615, 590, 570, 540, 520, 490, 465, 465};
31
32 uint16_t Lista_ADCD[27] = {3320, 2800, 2460, 2130, 1855, 1675, 1525, 1360, 1235, 1130, 1070, 990,
33                             940, 890, 840, 795, 770, 725, 695, 650, 625, 605, 575, 555, 555, 555};
34
35 typedef struct                               // Estructura para las muestras
36 {                                              // despues de conversion.
37     uint16_t chan1;                            //
38     uint16_t chan2;                            //
39     uint16_t chan3;                            //
40     uint16_t chan4;                            //
41 } MuestrasADC;                                //
42
43 typedef struct                               // Estructura para las muestras leidas
44 {                                              // del ADC.
45     uint32_t chan1;                            //
46     uint32_t chan2;                            //
47     uint32_t chan3;                            //
48     uint32_t chan4;                            //
49 } MuestrasLeidasADC;                        //

```

Figura 3.17. Tipos de datos del driver para el uso de los sensores de distancia

- **Lista_Distacia:** lista con las distancias, en centímetros, en las que detectan los sensores.
- **Lista_ADCF:** valores del sensor frontal correspondientes a las distancias de detección de éste.
- **Lista_ADCI:** valores del sensor izquierdo correspondientes a las distancias de detección de éste.
- **Lista_ADCD:** valores del sensor derecho correspondientes a las distancias de detección de éste.
- **MuestrasADC:** estructura que recoge las cuatro muestras del ADC una vez realizada la conversión de 32 bits a 16 bits.
- **MuestrasLeidasADC:** estructura que recoge las cuatro muestras del ADC.

Funciones:

```

38 /*****
39  * Funciones
40  *****/
41 void ADC_Init(void);           // Función para configurar los módulos ADC0 y ADC1.
42 uint8_t ReadFrontal();        // Función que lee el sensor frontal.
43 uint8_t ReadDerecha();        // Función que lee el sensor de la derecha.
44 uint8_t ReadIzquierda();      // Función que lee el sensor de la izquierda.
45 void ADC_DisparaF(void);      // Provoca el disparo de una conversión (ADC0, Secuenciador 1).
46 void ADC_DisparaD(void);      // Provoca el disparo de una conversión (ADC1, Secuenciador 1).
47 void ADC_DisparaI(void);      // Provoca el disparo de una conversión (ADC0, Secuenciador 2).
48 void ADC_Lee(uint16_t *datos); // Función para leer los datos obtenidos del ADC.
49 void ADC0S1IntHandler(void);  // Rutina de interrupción del ADC0 Secuenciador 1.
50 void ADC0S2IntHandler(void);  // Rutina de interrupción del ADC0 Secuenciador 2.
51 void ADC1S1IntHandler(void);  // Rutina de interrupción del ADC1 Secuenciador 1.

```

Figura 3.18. Funciones del driver para el uso de los sensores de distancia

- **ADC_Init():** Se habilitan los módulos ADC0, ADC1 y el puerto GPIOE, y su uso en bajo consumo. Se configuran los pines específicos para ser usados por los módulos ADC. Se configuran los secuenciadores 1, conectado al sensor frontal, y 2, conectado al sensor izquierdo, del módulo ADC0, y el secuenciador 1, conectado al sensor derecho, del módulo ADC1, con una velocidad de conversión máxima, disparo por software y la toma de cuatro muestras, siendo la última la que genere la interrupción. Por último, se limpian, habilitan y se da prioridad a las interrupciones de los secuenciadores, y se crea la cola de mensajes que se usa para almacenar los valores leídos de los secuenciadores.
- **ReadFrontal():** dispara el secuenciador conectado al sensor frontal, una vez su rutina de interrupción envía el valor a la cola, se recoge y en función del valor obtenido devuelve '1' en caso de detectar pared y '0' en caso de no detectarla.
- **ReadDerecha():** dispara el secuenciador conectado al sensor derecho, una vez su rutina de interrupción envía el valor a la cola, se recoge y en función del valor obtenido devuelve '1' en caso de detectar pared y '0' en caso de no detectarla.
- **ReadIzquierda():** dispara el secuenciador conectado al sensor izquierdo, una vez su rutina de interrupción envía el valor a la cola, se recoge y en función del valor obtenido devuelve '1' en caso de detectar pared y '0' en caso de no detectarla.
- **ADC_DisparaF():** activa el disparo software del secuenciador 1 del módulo ADC0.

- **ADC_DisparaD():** activa el disparo software del secuenciador 1 del módulo ADC1.
- **ADC_DisparaI():** activa el disparo software del secuenciador 2 del módulo ADC0.
- **ADC_Lee():** recoge el valor enviado a la cola de mensajes.
- **ADC0S1IntHandler():** rutina de interrupción del secuenciador 1 del módulo ADC0, hace una media de las cuatro muestras obtenidas y el valor resultante lo manda a la cola de mensajes.
- **ADC0S2IntHandler():** rutina de interrupción del secuenciador 2 del módulo ADC0, hace una media de las cuatro muestras obtenidas y el valor resultante lo manda a la cola de mensajes.
- **ADC1S1IntHandler():** rutina de interrupción del secuenciador 1 del módulo ADC1, hace una media de las cuatro muestras obtenidas y el valor resultante lo manda a la cola de mensajes.

Una aclaración es que, los valores llegan y salen de la cola de forma ordenada, ya que las lecturas de los sensores se hacen una tras otra y no de forma simultánea. Es decir, cuando se llama a la función de leer un sensor, inmediatamente después se llama a la función que recoge el valor de la cola y una vez tratado éste, se puede llamar a la lectura de otro sensor si se desea. De esta forma, un sensor no obtendrá un valor que no es el suyo y la cola de mensajes nunca se llenará.

3.2.1.3 Drivers para el uso del sensor brújula

Driver: *Brujula.c* *Brujula.h*

Defines:

```

15 /*****
16 * Defines
17 *****/
18 #define Brujula_ADDRESS      0x0D      // Direccion del sensor brujula
19 #define REGISTER_DATA_X_LSB  0x00      // Direccion del registro X parte baja
20 #define REGISTER_SIGNED_X_MSB 0x01      // Direccion del registro X parte alta
21 #define REGISTER_DATA_Y_LSB  0x02      // Direccion del registro Y parte baja
22 #define REGISTER_SIGNED_Y_MSB 0x03      // Direccion del registro Y parte alta
23 #define REGISTER_DATA_Z_LSB  0x04      // Direccion del registro Z parte baja
24 #define REGISTER_SIGNED_Z_MSB 0x05      // Direccion del registro Z parte alta
25 #define REGISTER_STATUS      0x06      // Direccion del registro de estado
26 #define REGISTER_TEMP_LSB    0x07      // Direccion del registro de temperatura parte baja
27 #define REGISTER_TEMP_MSB    0x08      // Direccion del registro de temperatura parte alta
28 #define REGISTER_CONTROL1     0x09      // Direccion del registro de control 1
29 #define REGISTER_CONTROL2     0x0A      // Direccion del registro de control 2
30 #define REGISTER_PERIOD       0x0B      // Direccion del registro periodo
31 #define DECLINACION           0.13      // Valor declinacion malaga
32 #define RAD_TO_DEG            ANG180/PI // Factor para conversion Rad->Deg
33 #define DESFASE               19        // Desfase
34 #define ANGULO_VARIACION       3        // Variacion angulo de 90
35 #define ANGULO_VARIACION180   20       // Variacion angulo de 180
36 #define ANG360                360       // Angulo de 360 grados
37 #define ANG269                269       // Angulo de 269 grados
38 #define ANG180                180       // Angulo de 180 grados
39 #define ANG90                 90        // Angulo de 90 grados

```

Figura 3.19. Defines del driver para el uso del sensor brújula

Funciones:

```

42 /*****
43 * Funciones
44 *****/
45 void Brujula_Init();           // Configuracion el sensor brújula.
46 void Brujula_Giro90(uint8_t giro); // Funcion para realizar giros de 90 grados.
47 void Brujula_Giro180();        // Funcion para realizar giros de 180 grados.
48 uint16_t Brujula_CalcularGrados(); // Funcion para calcular el angulo respecto del norte
49                                // en funcion de las coordenadas X e Y.
50 void Brujula_ReadData(int16_t* x, int16_t* y, int16_t* z); // Función encargada de leer la brújula.

```

Figura 3.20. Funciones del driver para el uso del sensor brújula

- **Brujula_Init():** configura el sensor brújula para que tenga un modo de operación continuo, una frecuencia de actualización de datos de 200 Hz, una escala de sensibilidad de ± 8 Gauss y una tasa de sobremuestreo de 512. Para ello, se hace uso de la comunicación I2C y se escribe el valor en el registro de control 1. Por último, se establece el tiempo de las operaciones SET/RESET recomendado por el fabricante.
- **Brujula_Giro90():** se encarga de realizar giros de 90 grados, para la derecha o izquierda, según se indique. Para ello, se obtiene en primer

lugar el ángulo en el que se encuentra el microbot y seguidamente se calcula el ángulo final deseado. Luego, se hace girar al microbot en la dirección indicada hasta conseguir el ángulo deseado. Un cambio al modo inactivo o un reset, supone la salida del bucle que realiza dicho giro.

- **Brujula_Giro180():** se encarga de realizar giros de 180 grados, de igual forma que en el caso de ángulos de 90 grados.
- **Brujula_ReadData():** lee los registros correspondientes a los ejes X, Y, Z del sensor.
- **Brujula_CalcularGrados():** calcula el ángulo respecto del norte magnético en función de las coordenadas X e Y, y devuelve dicho.

3.2.1.4 Driver para la comunicación UART

Driver: *ComUART.c* *ComUART.h*

Funciones:

```

16 /*****
17 *  Funciones
18 *****/
19 void UART2_Init(void);           // Configuración del módulo UART2.
20 void UART_Task(void *pvParameters); // Tarea encargada de la gestión
21                                     // de comandos recibidos.
22 void UART_RecvData(uint32_t ui32UARTBase, uint8_t* dato); // Función para recibir por la UART
23 void UART_SendData(uint32_t ui32UARTBase, unsigned char *pui8Buffer, // Función para enviar por la UART
24                     uint32_t ui32Count); //
25 void UART2IntHandler(void);      // Rutina de interrupción del módulo UART2

```

Figura 3.21. Funciones del driver encargado de la comunicación UART

- **UART2_Init():** configura el módulo UART2, encargado de comunicarse con el microcontrolador ESP32. Para ello, se habilita el uso del módulo UART2 y el puerto GPIOD, y su uso en bajo en consumo. Se seleccionan los pines PD6, como receptor, y PD7, como transmisor, para el uso del módulo UART2, es necesario desbloquear el pin PD7 para su uso. Se habilitan las interrupciones del módulo, tanto en la recepción como en el *timeout* de recepción, y finalmente se habilita el módulo UART2. Por último, se crea la cola de mensajes que se usa en el driver.
- **UART_Task():** esta tarea se encarga de la gestión de los comandos recibidos por parte de la aplicación Qt. Dicha tarea queda a la espera de la llegada de los comandos a la cola de mensajes, una vez recibidos, dependiendo del comando, ejecuta la lógica oportuna.

- **UART_RecvData():** función encargada de la recepción de la información transmitida al módulo UART.
- **UART_SendData():** función encargada de la transmisión de información en el módulo UART.
- **UART2IntHandler():** rutina de interrupción encargada de recoger la información transmitida al módulo UART y enviarlo a la cola de mensajes.

3.2.1.5 Driver para la comunicación I2C

Driver: *ComI2C.c* *ComI2C.h*

Funciones:

```

8/*****
9 * Funciones
10 *****/
11 void I2C1_Init(void);           // Configuración del modulo I2C1
12 void I2C1_Write(uint8_t slave_addr, uint8_t reg_addr, uint8_t data); // Función encargada de escribir en el bus I2C
13 uint8_t I2C1_Read(uint8_t slave_addr, uint8_t reg_addr);           // Función encargada de leer en el bus I2C

```

Figura 3.22. Funciones del driver para la comunicación I2C

- **I2C1_Init():** se habilita el uso, tanto normal como en bajo consumo, del módulo I2C1 y el puerto GPIOA. Se configuran los pines PA6, responsable de la señal de reloj, y PA7, responsable de la señal de datos, para su utilización por parte del módulo I2C1 y se configura una resistencia *pull-up* software para cada uno de ellos. Por último, se inicializa el bloque master del módulo.
- **I2C1_Write():** función encarga de escribir en el bus I2C, asegurando un único uso del bus al mismo tiempo.
- **I2C1_Read():** función encargada de leer en el bus I2C, asegurando un único uso del bus al mismo tiempo.

3.2.1.6 Driver para la navegación

Driver: *Navegacion.c* *Navegacion.h*

Defines:

```
17 /*****
18 * Defines
19 *****/
20 #define MAPA_SIZE 21 // Tamaño del mapa 21x21.
21 #define MSG_SIZE 13 // Tamaño del mensaje.
22 #define DISTANCIA 20 // Distancia en cm que recorre el robot para recoger informacion.
23 #define R 3.5 // Radio de la rueda en cm
24 #define N_GIROS (12*DISTANCIA)/2*M_PI*R // Nº giros para la distancia deseada.
25 #define CENTRO 10 // Posicion del centro (10,10).
26
27 #define ENCODER_I GPIO_PIN_4 // Etiqueta con el pin del sensor encoder.
28 #define SENSOR_BAJO GPIO_PIN_6 // Etiqueta con el pin del sensor bajo.
```

Figura 3.23. Defines del driver para la navegación

Tipos de datos:

```
29 typedef enum // Tipo de dato para referirse
30 { // al estado de las celdas
31     DESCONOCIDO, // en las que se divide
32     CONOCIDO, // el mapa.
33     X //
34 } EstadoCelda_t; //
35
36 typedef enum // Tipo de dato para referirse
37 { // a la existencia de pared o no.
38     NO_PARED, //
39     PARED //
40 } Pared_t; //
41
42 typedef enum // Tipo de dato para referirse
43 { // al sentido de movimiento.
44     NEGATIVO, //
45     POSITIVO //
46 } Sentido_t; //
47
48 typedef enum // Tipo de dato para referirse
49 { // al eje de movimiento.
50     HORIZONTAL, //
51     VERTICAL //
52 } Eje_t; //
53
54 typedef struct // Estructura para referirse
55 { // a la orientacion del microbot
56     Sentido_t sentido; // dentro del laberinto
57     Eje_t eje; //
58 } Orientacion_t; //
59
60 typedef struct // Estructura para referirse
61 { // a las paredes del laberinto.
62     Pared_t d; //
63     Pared_t i; //
64     Pared_t f; //
65 } Paredes_t; //
66
67 typedef struct // Estructura que engloba toda
68 { // la informacion referente a
69     int8_t x; // la posicion del microbot
70     int8_t y; // dentro del laberinto.
71     Orientacion_t orientacion; //
72     Paredes_t paredes; //
73     EstadoCelda_t estado; //
74 } InfoPosicion_t; //
```

Figura 3.24. Tipos de datos usados en el driver para la navegación

- **EstadoCelda_t**: se refiere al estado de una celda, este estado puede ser DECONOCIDO, en caso de que el microbot no haya pasado nunca por esa celda, CONOCIDO, en caso de que haya pasado y hubiera varios caminos posibles, X, en caso de que el microbot no deba pasar por esa celda, principalmente para cerrar caminos ya recorridos y evitar repetirlos.
- **Pared_t**: se refiere a la existencia de una pared en la celda en la que se encuentre el microbot, puede ser NO_PARED, en caso de no detectar una pared en la celda correspondiente, o PARED, en el caso de si detectar una pared en dicha celda.
- **Sentido_t**: se refiere al sentido de movimiento del microbot, haciendo similitud con un eje de coordenadas, NEGATIVO, sería en caso de desplazarse en dirección hacia los ejes negativos, y POSITIVO, en caso de dirigirse hacia a los ejes positivos.
- **Eje_t**: se refiere al tipo de movimiento que realiza el microbot, haciendo similitud, igual que en el caso anterior, con un eje de coordenadas, HORIZONTAL, sería un movimiento sobre el eje X, y VERTICAL, un movimiento sobre el eje Y.
- **Orientacion_t**: agrupa el sentido y eje de movimiento del microbot.
- **Paredes_t**: agrupa las paredes posibles a detectar, frontal, derecha e izquierda.
- **InfoPosicion_t**: agrupa toda la información referente a la posición del microbot en una celda dentro del laberinto.

Funciones:

```

84 /*****
85 *  Funciones
86 *****/
87 void NavegacionInit();           // Función para configurar el encoder y sensor bajo e
88                                 // inicializar el mapa y la posición del robot en el mapa.
89 void ResetMapa();               // Función para resetear el mapa y restablecer los valores de inicio.
90 void ActualizarMapa();          // Función encargada de actualizar el mapa con la posición del robot
91                                 // y la información recogida por este.
92 void ActualizarPosicion();       // Función encargada de actualizar la posición del robot.
93 void Navegacion_Task(void *pvParameters); // Tarea encargada de la navegación del robot en el laberinto.
94 void Enviar_mensaje();          // Función encargada de construir y enviar el mensaje por la UART.
95 void Decidir_Movimiento();       // Función encargada de decidir el próximo movimiento.
96 void OnOffEncoders(bool on_off); // Función para habilitar o deshabilitar el encoder.
97 void GPIOIntHandle(void);       // Rutina encargada de las interrupciones del encoder, sensor_bajo.

```

Figura 3.25. Funciones del driver para la navegación

- **NavegacionInit():** habilita el puerto GPIOB, su uso en bajo consumo y las interrupciones de dicho puerto. Se configuran los pines usados, por el sensor encoder, PB4, y el sensor bajo, PB6, como entradas GPIO. Se habilitan sus interrupciones y cuando éstas se producen, en caso del sensor encoder, es en ambos flancos, y del sensor bajo, en flanco de bajada. Luego, se crea el semáforo que da inicio al algoritmo de navegación implementado. Por último, se inicializa el mapa con todas sus celdas a desconocidas, la posición del microbot en el centro del mapa, su orientación como vertical y positiva, así como ciertas variables globales usadas en la implementación.
- **ResetMapa():** restablece los valores de inicio en el mapa, posición y orientación del microbot, y las variables globales utilizadas.
- **ActualizarMapa():** carga la información obtenida por el microbot en la celda del mapa correspondiente.
- **ActualizarPosicion():** actualiza las coordenadas del microbot una vez recorrida la distancia deseada.
- **Navegacion_Task():** esta tarea constituye el núcleo principal del algoritmo de navegación. Dicha tarea queda a la espera de la activación del semáforo que da paso al algoritmo. Primeramente, una vez iniciado el algoritmo, pero no el movimiento, el microbot hace una primera toma de información del entorno, entonces, actualiza esa primera celda del mapa, crea el mensaje con dicha información y lo envía a través de la UART, y decide el movimiento en función a esa información. Después, a medida que el microbot vaya recorriendo la distancia establecida, repite el mismo proceso, actualiza el mapa con la información de la celda correspondiente, crea y envía el mensaje, y decide el próximo movimiento, hasta llegar al final del laberinto, donde interrumpe el sensor bajo y finaliza el movimiento.
- **Enviar_mensaje():** crea el mensaje con toda la información de la posición del microbot y lo envía por la UART. Este mensaje se crea de la siguiente forma: X.Y.R.L.F.S.E, donde X es la coordenada x, Y es la coordenada y, R es la pared derecha, L es la pared izquierda, F es la pared frontal, S es

el sentido de movimiento, E el eje de movimiento. Cada campo separado por un punto ('.'), sin punto al inicio ni al final.

- **Decidir_Movimiento():** función que se encarga de decir el movimiento del microbot en función de los caminos disponibles. El microbot siempre avanza hacia el frente hasta encontrar una pared frontal, cuando esto ocurra siempre girará hacia la derecha, en caso de poderse, si no se puede girará hacia la izquierda, o en caso de una calle sin salida retrocederá.
- **OnOffEncoders():** activa o desactiva el sensor encoder, para que cuando el microbot gira, no cuente esos giros como desplazamiento.
- **GPIOIntHandle():** rutina de interrupción del sensor encoder y el sensor suelo. En el caso de interrumpir el sensor encoder, si se han realizado los giros correspondientes a la distancia establecida, actualiza la posición del microbot y activa el semáforo que inicia el algoritmo de navegación; en caso de interrumpir el sensor bajo, finaliza el movimiento y realiza un reset.

3.2.1.7 Archivo Tiva_common

Este archivo recoge todas las librerías básicas utilizadas en el proyecto, así como las necesarias para el uso del sistema operativo FreeRTOS. De igual forma, se definen los tipos de datos usados para los comandos, estado y modo del microbot, véase la Figura 3.26.


```

44 typedef enum          // Comandos:
45 {
46     CMD_MODALDLE = 0,  // Modo Inactivo
47     CMD_MODALMANUAL,   // Modo Manual
48     CMD_MODALAUTOMATICO, // Modo Automático
49     CMD_PARADA,        // Parada
50     CMD_AVANCE,        // Avance
51     CMD_GIRO_D,        // Giro hacia la derecha
52     CMD_GIRO_I,        // Giro hacia la izquierda
53     CMD_RETROCESO,     // Retroceso
54     CMD_START_AUTO,    // Inicio algoritmo de navegacion
55     CMD_STOP_AUTO,     // Detención del algoritmo de navegacion
56     CMD_CONTINUE_AUTO, // Continuar el algoritmo de navegacion
57     CMD_GIRO_90D,      // Giro de 90 grados hacia la derecha
58     CMD_GIRO_90I,      // Giro de 90 grados hacia la izquierda
59     CMD_RESET          // Reset
60 } cmd_accion_t;
61
62
63 typedef enum          // Estado del robot:
64 {
65     PARADO = 0,        // Parado
66     GIRANDO,           // Girando
67     AVANZANDO,         // Avanzando
68     RETROCEDIENDO      // Retrocediendo
69 } rbt_estado_t;
70
71
72 typedef enum          // Modo de funcionamiento:
73 {
74     IDLE = 0,          // Modo Inactivo
75     MANUAL,            // Modo Manual
76     AUTOMATICO         // Modo Automático
77 } rbt_modos_t;
78

```

Figura 3.26. Tipos de datos para comandos, estado y modo del microbot

3.2.1.8 Archivo tm4c123gh6pm_startup_ccs

Este archivo es generado por el entorno de desarrollo y su función es asignarle a cada módulo la rutina de interrupción que se desea que ejecute dicho módulo. Por tanto, se han incluido las rutinas de interrupción ya comentadas en los distintos drivers implementados.

3.2.1.9 Archivo main

En términos generales, es el archivo principal del proyecto. En él, se crean las variables globales referidas al modo y estado del microbot. Se selecciona el reloj del sistema y su frecuencia, se hace llamada a las inicializaciones de los drivers implementados y se crean las tareas usadas en ellos. Por último, se inicia el sistema operativo FreeRTOS.

3.2.2 ESP32-WROOM-32

El software implementado en este microcontrolador se encarga de crear la pasarela entre el microcontrolador TIVA y la aplicación Qt, al contar con un módulo Wifi que permite incorporar al microbot en el Internet de las Cosas. Para dicha implementación se hace uso del entorno de desarrollo Visual Studio Code [17]. De igual forma, como apoyo se han consultado ejemplos compartidos por el fabricante [18] para el desarrollo de este software.

La implementación se recoge en un único archivo, dividido en varias partes, que corresponden con los módulos UART y Wifi, el cliente MQTT y la parte main del archivo.

Archivo: *app_main.c*

Defines:

```

/*-----*
*                               *
*                               *
*-----*/

#define CMD_MODE_IDLE           0
#define CMD_MODE_MANUAL        1
#define CMD_MODE_AUTO          2
#define RX_PACK_SIZE           13
#define BROKER                  "mqtt://broker.emqx.io"
#define PORT                    1883
#define LOC_TOPIC               "Localizacion/Coordenadas"
#define DAT_TOPIC               "Info/Comando"
#define ACT_TOPIC               "Info/Accion"
#define INF_TOPIC               "Info/Informacion"
#define TXD_PIN                 (GPIO_NUM_17)
#define RXD_PIN                 (GPIO_NUM_16)
#define WIFI_SSID                "Luismimovil"
#define WIFI_PASS                "luismi123"
#define WIFI_CONN_MAX_RETRY     6
#define WIFI_SCAN_METHOD        WIFI_ALL_CHANNEL_SCAN
#define WIFI_CONNECT_AP_SORT_METHOD  WIFI_CONNECT_AP_BY_SIGNAL
#define WIFI_SCAN_RSSI_THRESHOLD -127
#define WIFI_SCAN_AUTH_MODE_THRESHOLD  WIFI_AUTH_WPA2_PSK
#define NETIF_DESC_STA          "Netif_STA"
#define PREFERRED_IPV6_TYPE     ESP_IP6_ADDR_IS_LINK_LOCAL

```

Figura 3.27. Defines del driver implementado en el ESP32

Funciones:

Módulo UART:

```

/*-----*
*                               *
*                               *
*-----*/

/**
 * Funcion para inicializar el modulo UART
 */
void UART_Init(void)...

/**
 * Funcion encargada de enviar datos por la UART
 * @return N° de bytes enviados
 */
int sendData(const uint8_t* data)...

/**
 * Tarea encargada de la recepcion de datos por la UART
 */
static void rx_task(void *arg)...

```

Figura 3.28. Funciones para el uso del módulo UART del ESP32

- **UART_Init():** se inicializa el módulo UART con la configuración correspondiente para que permita la comunicación con el módulo UART del microcontrolador TIVA.
- **sendData():** función encargada de enviar datos por la UART.
- **rx_task():** tarea encargada de la recepción de datos por la UART.

Cliente MQTT:

```
/*-----*
 *                               *
 *                               *
 *-----*/

/**
 * Funcion que maneja los eventos recibidos de MQTT
 */
static void mqtt_event_handler(void *handler_args, esp_event_base_t base,
                               int32_t event_id, void *event_data) ...

/**
 * Funcion para inicializar el cliente MQTT
 */
void MQTT_Init(void) ...
```

Figura 3.29. Funciones del cliente MQTT del ESP32

- **mqtt_event_handler():** función que maneja los eventos recibidos del cliente MQTT.
- **MQTT_Init():** crea el cliente MQTT, lo conecta al *broker* y registra el manejador de eventos para el cliente.

Módulo Wifi:

```
/*-----*
 *                               *
 *                               *
 *-----*/

/**
 * Funcion para manejar los eventos recibidos del modulo WIFI
 */
static void wifi_event_handler(void* arg, esp_event_base_t event_base,
                               int32_t event_id, void* event_data) ...

/**
 * Funcion para desconectar, limpiar los manejadores de eventos y
 * restaurar por defecto el modulo WIFI
 */
void wifi_disconnect(void) ...

/**
 * Funcion para inicializar el modulo WIFI
 */
void WIFI_Init(void) ...
```

Figura 3.30. Funciones para el uso del módulo Wifi del ESP32

- **wifi_event_handler():** función que maneja los eventos recibidos del módulo Wifi.
- **wifi_disconnect():** función para desconectar, limpiar los manejadores de eventos y restaurar por defecto el módulo Wifi.
- **WIFI_Init():** inicializa y configura el módulo Wifi en modo STA, modo estación, que permite conectarse a una red Wifi existente.

En cuanto a la parte main del archivo:

- Se realiza una partición de la memoria flash, destinada al proyecto, de forma que no se pierde información tras un reinicio o apagado del dispositivo.
- Se inicializa el *stack* de red del ESP32, es decir, se preparan las estructuras internas necesarias para manejar interfaces de red, como en este proyecto, el Wifi.
- Se crea e inicializar un bucle de eventos globales por defecto, el cual es el mecanismo encargado de la gestión de los eventos del sistema.
- Se hace llamada a las inicializaciones de los módulos UART y Wifi, y del cliente MQTT.
- Se configura el pin GPIO2 como salida, para el control del led.
- Se crea la tarea *rx_task*, mencionada en la parte del módulo UART.

3.2.3 Aplicación Qt

En este apartado, se describe detalladamente el software implementado para la creación de la aplicación, encargada de controlar el microbot mediante un cliente MQTT. Para el desarrollo de dicha aplicación, se hace uso de Qt Creator, un entorno de desarrollo integrado usado principalmente para crear aplicaciones gráficas multiplataforma con Qt y con un *framework* en C++ [19]. Para la obtención del entorno de desarrollo, también se puede hacer uso de una licencia educativa mediante las credenciales de la universidad.

La aplicación permite alternar entre los modos de funcionamiento del microbot, Figura 3.31, controlarlo manualmente y dar comienzo a su algoritmo de navegación mostrando gráficamente su movimiento y entorno.

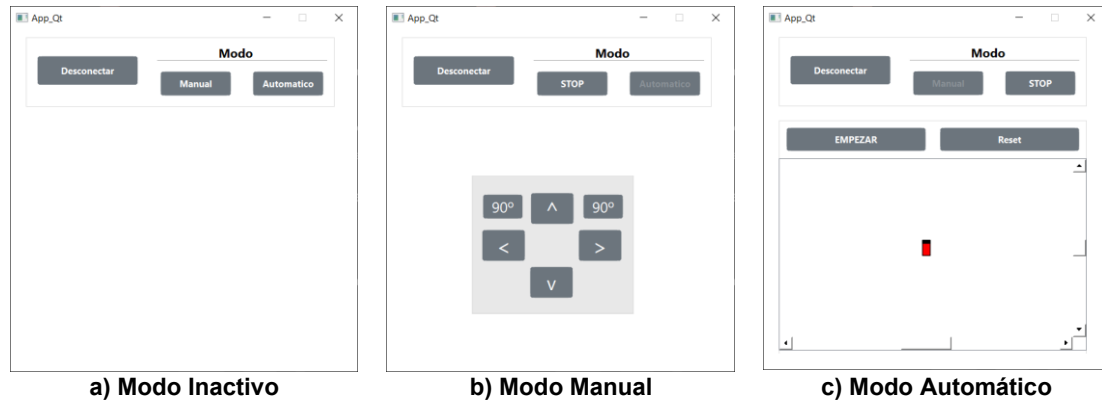


Figura 3.31. Modos de funcionamiento del microbot

El software implementado se divide, principalmente, en dos drivers: *mainwindow* y *robotwidget*. Ambos drivers consisten en la creación de una clase y la implementación de las funciones de dicha clase.

En el caso del driver *mainwindow*, se implementa la funcionalidad de los widgets incorporados en la interfaz de usuario, con excepción del gráfico donde se representa el movimiento y entorno del microbot, y la creación del cliente MQTT y el manejo de las señales de éste, Figura 3.32.

En el caso del driver *robotwidget*, se encarga únicamente de la funcionalidad del gráfico que muestra el movimiento y entorno del microbot, Figura 3.33.

La comunicación de la aplicación es la siguiente:

- La aplicación envía hacia el microbot el comando correspondiente a la acción del widget con el que se interactúa, a través del cliente MQTT suscrito al *topic* "Info/Comando".
- La aplicación recibe desde el microbot la información correspondiente al desplazamiento y entorno de éste, mediante el cliente MQTT, el cual está suscrito al *topic* "Navegacion/Info".

```
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

public slots:
    void suscribirTopics();           // Función para suscribirse a los topics
private slots:

    //Mqtt
    void on_BConnect_Disconnect_clicked(); /*Boton de conectar/desconectar MQTT pulsado*/
    void brokerConnected_Disconnected(); /*Slot correspondiente a la señal de conexion/desconexion MQTT*/

    //Modo Manual
    void on_BManual_clicked();           /*Boton de Activacion del modo Manual pulsado*/
    void on_BAvance_pressed();           /*Boton Avance presioando*/
    void on_BAvance_released();          /*Boton Avance soltado*/
    void on_BGiro_I_pressed();            /*Boton Giro Izquierda presionado*/
    void on_BGiro_I_released();           /*Boton Giro Izquierda soltado*/
    void on_BRetroceso_pressed();         /*Boton Retroceso pulsado*/
    void on_BRetroceso_released();        /*Boton Retroceso soltado*/
    void on_BGiro_D_pressed();            /*Boton Giro Derecha presionado*/
    void on_BGiro_D_released();           /*Boton Giro Derecha soltado*/
    void on_BGiro_D90_clicked();          /*Boton Giro 90 grados Derecha pulsado*/
    void on_BGiro_I90_clicked();          /*Boton Giro 90 grados Izquierda pulado*/

    //Modo Automatico
    void on_BAutomatic_clicked();         /*Boton de Activacion del modo Automatico pulsado*/
    void on_Start_Auto_clicked();         /*Boton de comienzo de movimiento automatico pulsado*/
    void on_Restore_clicked();            /*Boton para resetear el grafico de recorrido*/

signals:

private:
    Ui::AppQt *ui;
    QMqttClient *m_client;

    void CentrarRobot(int x, int y);      // Función encargada de centrar el robot en la vista del gráfico
};
```

Figura 3.32. Clase *mainwindow*

```
class RobotWidget : public QWidget
{
    Q_OBJECT

public:
    RobotWidget(QWidget *parent = nullptr);
    void restoreFault();           // Función encargada de restaurar por defecto el gráfico y las estructuras usadas

protected:
    void paintEvent(QPaintEvent *event) override;           // Funcion que se encargada de dibujar en el gráfico

public slots:
    void onMessageReceived(const QByteArray &message,      // Maneja los mensajes recibidos de MQTT
                           const QMqttTopicName &topic);
private slots:

signals:
    void CRobot(int x, int y);           // Señal emitida para con la posicion del robot en gráfico para centrar la vista

private:
    QPoint robotPosition;                // Posición actual del robot
    QList<QPoint> path;                  // Almacena el recorrido del robot
    QList<int> wallInfo;                 // Almacena la informacion de las paredes [Derecha, Izquierda, Frente]
    QSet<Wall> paintedWalls;            // Contiene las paredes ya pintadas
    int orientInfo;                     // Almacena la informacion de la orientacion
    int sentidoInfo;                    // Almacena la informacion del sentido de movimiento
    bool isFirstUpdate;                 // Indica si es la primera actualización
};
```

Figura 3.33. Clase *robotwidget*

Capítulo 4. Verificación y pruebas

4.1 Sistema de pruebas

Los sistemas de pruebas realizados son similares en los tres núcleos que constituyen el proyecto, siendo estos núcleos: microcontrolador TIVA, microcontrolador ESP32 y aplicación Qt.

En cuanto al microcontrolador TIVA, el sistema de pruebas se ha basado en una tarea, la cual integra un bucle infinito del que nunca se sale, y el uso del módulo UART0 para la comunicación con un terminal del entorno de desarrollo y poder visualizar información que se desee.

Para la parte del microcontrolador ESP32, se hace uso de la librería *esp_log.h*, la cual permite visualizar, en el terminal del entorno, la información que se indique.

En el caso de la aplicación Qt, se hace uso de una librería interna de Qt, llamada *Qdebug*, la cual permite añadir comentarios que pueden ser visualizados a través del terminal de entorno de desarrollo. Además, para comprobar el flujo de información del cliente MQTT, se utiliza la herramienta cliente MQTTX [20], que permite probar, depurar y visualizar las comunicaciones basadas en este protocolo.

4.2 Pruebas realizadas

En primer lugar, se realizan pruebas para comprobar el correcto funcionamiento de los componentes hardware que conforman el microbot y los módulos utilizados para el manejo de estos componentes.

- Requisito 1.1.1:

Este requisito se divide en dos requisitos, 1.1.1.1 y 1.1.1.2, que corresponden a los motores y el sensor brújula, respectivamente.

Puesto que estos componentes son controlados por el microcontrolador TIVA, se hace uso del sistema de pruebas implementado en este microcontrolador. Dichas pruebas consisten en el uso de las funciones implementadas, en el driver correspondiente a cada componente, para el uso de estos.

Por tanto, en caso de los motores, requisito 1.1.1.1, se hace llamada, dentro la tarea de pruebas, a cada función que controla el movimiento y se hace una espera entre cada función, de esta forma, se puede apreciar cada tipo de movimiento durante un tiempo, como se muestra en la Figura 4.1.

```
48 void Prueba_Task(void *pvParameters)
49 {
50     while(1)
51     {
52         Avanzar();
53         vTaskDelay(pdMS_TO_TICKS(5000));
54         Retroceder();
55         vTaskDelay(pdMS_TO_TICKS(5000));
56         Parar();
57         vTaskDelay(pdMS_TO_TICKS(5000));
58         Girar_Drc();
59         vTaskDelay(pdMS_TO_TICKS(5000));
60         Girar_Izq();
61         vTaskDelay(pdMS_TO_TICKS(5000));
62         GirarLento_Drc();
63         vTaskDelay(pdMS_TO_TICKS(5000));
64         GirarLento_Izq();
65         vTaskDelay(pdMS_TO_TICKS(5000));
66     }
67 }
68 }
```

Figura 4.1. Prueba del requisito 1.1.1.1

Obteniendo un correcto funcionamiento por parte de los motores y cumpliendo con el requisito 1.1.1.1.

Para el caso del sensor brújula, requisito 1.1.1.2, se procede con el mismo mecanismo, pero en esta ocasión se hace uso de las funciones del driver correspondiente a dicho sensor, las cuales permiten al microbot realizar giros de 90 grados en ambos sentidos y de 180 grados, que se realiza siempre hacia la derecha, como se muestra en la Figura 4.2.

```
48 void Prueba_Task(void *pvParameters)
49 {
50     while(1)
51     {
52         Brujula_Giro90(CMD_GIRO_90D);
53         Brujula_Giro90(CMD_GIRO_90I);
54         Brujula_Giro180();
55     }
56 }
```

Figura 4.2. Prueba del requisito 1.1.1.2

En este caso, se puede apreciar que no hay esperas, puesto que las mismas funciones constituyen un bucle, el cual no termina hasta realizar el giro correspondiente. De esta prueba, se obtiene un correcto funcionamiento de las funciones, así como de la comunicación I2C entre el microcontrolador y el sensor y, por tanto, satisfaciendo el requisito 1.1.1.2.

Por lo que, con el cumplimiento de ambos requisitos, desemboca en el cumplimiento del requisito 1.1.1.

- Requisito 1.1.2:

Este requisito concierne el uso de los sensores de distancia, en el ejercicio de este proyecto, su función es detectar, dentro del rango de medida de los sensores, la existencia de paredes, o no, del laberinto.

Para ello, se debe comprobar el correcto funcionamiento del driver implementado para estos sensores, así como de los módulos ADC utilizados en él.

Por tanto, para la comprobación, se añaden las funciones encargadas de leer los sensores en la tarea de prueba, y se hace uso del módulo UART conectado al terminar del entorno de desarrollo para comprobar tal funcionamiento, como se muestra en la Figura 4.3.

```

48 void Prueba_Task(void *pvParameters)
49 {
50     uint8_t f, d, i;
51
52     while(1)
53     {
54         f = ReadFrontal();
55         d = ReadDerecha();
56         i = ReadIzquierda();
57
58         UARTprintf("Sensor Frontal: %d\nSensor Derecho: %d\nSensor Izquierdo: %d\n\n", f, d, i);
59         vTaskDelay(pdMS_TO_TICKS(5000));
60     }
61 }

```

Figura 4.3. Prueba del requisito 1.1.2

Estas funciones están implementadas para que devuelvan '0', en caso de no detectar una pared, y '1', en caso de detectarla.

Por lo que, un ejemplo de ello es colocar una pared frente a algún sensor y dejar libre los otros, como se muestra en la Figura 4.4.

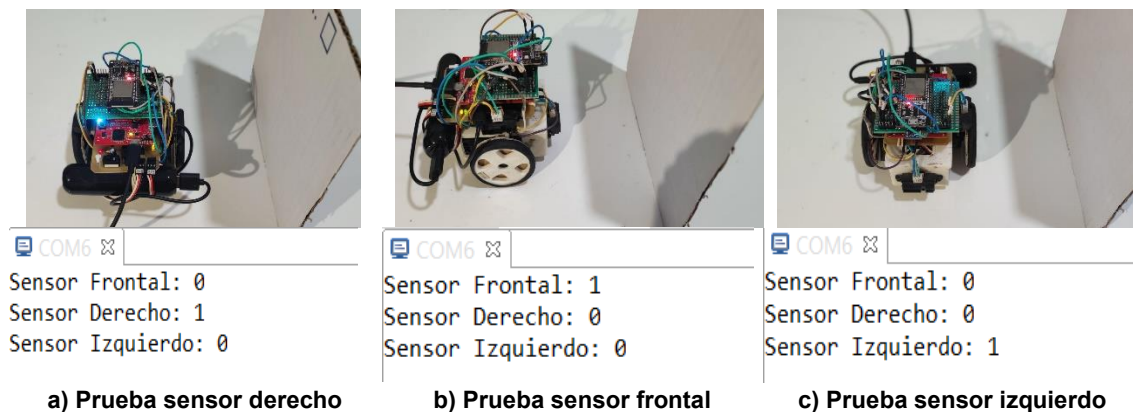


Figura 4.4. Ejemplo de prueba del requisito 1.1.2

Tras el ejemplo mostrado, se puede apreciar el correcto funcionamiento del driver implementado para los sensores de distancia y, por tanto, el cumplimiento del requisito 1.1.2.

- Requisito 1.1.3:

Este requisito es uno de lo más importantes del proyecto, ya que recae sobre éste el peso de la comunicación entre los núcleos que componen el proyecto. Se divide en dos requisitos, por un lado, la comunicación entre el microcontrolador TIVA y el ESP32, requisito 1.1.3.1, y, por otro lado, la comunicación entre el microcontrolador ESP32 y el cliente MQTT, requisito 1.1.3.2.

Ambos requisitos se pueden comprobar a la vez, ya que para que el microcontrolador TIVA publique el mensaje en el cliente MQTT, el mensaje debe

pasar por el microcontrolador ESP32, que será quien realmente publique el mensaje en el cliente MQTT.

La prueba realizada es la siguiente, se crea un mensaje en el microcontrolador TIVA y éste se transmite al microcontrolador ESP32 a través del módulo UART, una vez recibido por el ESP32, lo publica en el *topic* correspondiente.

El mensaje ejemplo tiene la estructura del mensaje que envía el microbot cuando analiza el entorno, por lo que, el ESP32 debe recibir y publicar en el *topic* correspondiente el mismo mensaje que envió el microcontrolador TIVA. Para una mejor comprensión, se explica cuál es el mensaje: el microbot se encuentra en las coordenadas (1,1), tiene paredes a la derecha y a la izquierda, y su movimiento tiene sentido positivo y vertical, por lo que el mensaje tiene la forma “1.1.1.1.0.1.1”. Para observar el flujo del mensaje, se hace uso del terminal del entorno de desarrollo utilizado para la programación del microcontrolador ESP32, Visual Studio Code, y la herramienta cliente MQTTX.

En primer lugar, se muestra en la Figura 4.5, la creación y envío del mensaje por parte del microcontrolador TIVA.

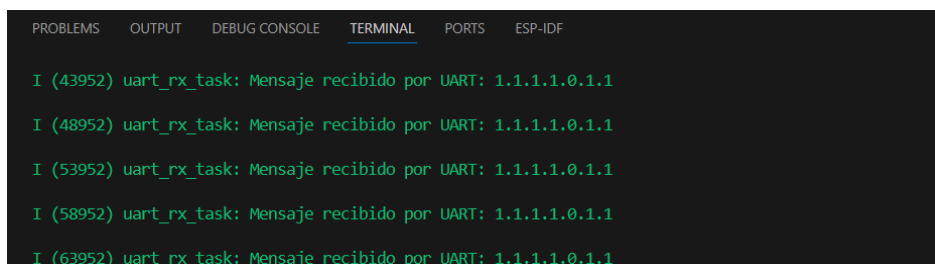
```

48 void Prueba_Task(void *pvParameters)
49 {
50
51     unsigned char* mensaje = "1.1.1.1.0.1.1";
52
53     while(1)
54     {
55         UART_SendData(UART2_BASE, mensaje, strlen((const char*)mensaje)+1);
56         vTaskDelay(pdMS_TO_TICKS(5000));
57     }
58 }

```

Figura 4.5. Prueba del requisito 1.1.3

Seguidamente, en la Figura 4.6, se muestra la recepción, por parte del microcontrolador ESP32, del mensaje ejemplo.



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF
I (43952) uart_rx_task: Mensaje recibido por UART: 1.1.1.1.0.1.1
I (48952) uart_rx_task: Mensaje recibido por UART: 1.1.1.1.0.1.1
I (53952) uart_rx_task: Mensaje recibido por UART: 1.1.1.1.0.1.1
I (58952) uart_rx_task: Mensaje recibido por UART: 1.1.1.1.0.1.1
I (63952) uart_rx task: Mensaje recibido por UART: 1.1.1.1.0.1.1

```

Figura 4.6. Prueba del requisito 1.1.3.1 en el ESP32

Y, por último, la publicación de dicho mensaje en el *topic* correspondiente, por parte del ESP32, Figura 4.7.

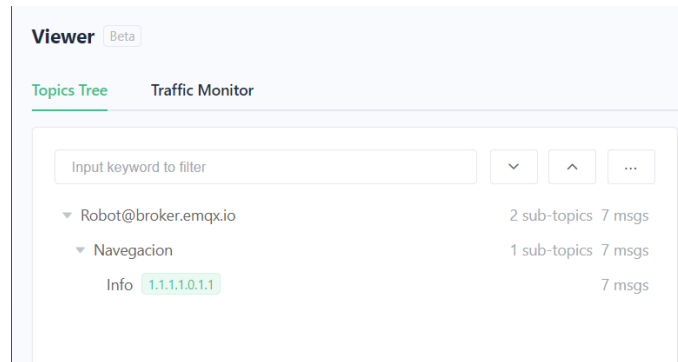


Figura 4.7. Prueba del requisito 1.1.3.2 en el cliente MQTT

Con la comprobación del anterior ejemplo, se puede corroborar el cumplimiento de los requisitos 1.1.3.1 y 1.1.3.2, y, en consecuencia, del requisito 1.1.3.

- Requisito 1.1.4:

Este requisito está orientado a comprobar el correcto funcionamiento del sensor bajo del microbot, el cual se encarga de dar por finalizado el movimiento y el algoritmo de navegación, una vez el microbot complete el laberinto. Es sencillo de comprobar, basta con hacer avanzar al microbot sobre una superficie clara, principalmente de color blanco, hasta llegar a otra superficie de una tonalidad más oscura, preferiblemente de color negro. De esta forma, mediante la interrupción del sensor, detiene el movimiento y se realiza un reseteo de la lógica usada para la navegación, dejando preparado al microbot para realizar nuevos intentos. Tras la prueba realizada, se comprueba un correcto funcionamiento de la lógica implementada para el sensor bajo, y, por consiguiente, el cumplimiento del requisito 1.1.4.

Con el cumplimiento de los requisitos 1.1.1, 1.1.2, 1.1.3 y 1.1.4, queda corroborado el cumplimiento del requisito 1.1, el cual consiste en que el microbot debe ser capaz de desplazarse por el laberinto y comunicarse con el cliente MQTT.

- Requisito 1.2.1:

Este requisito concierne a la aplicación Qt, se divide en dos requisitos, por un lado, la aplicación debe poder controlar el microbot, es decir, enviar comandos al cliente MQTT, que corresponde con el requisito 1.2.1.1, y, por otro lado, debe poder monitorizar el movimiento del microbot, lo que significa que debe poder recibir la información transmitida al cliente MQTT desde el microbot, que corresponde con el requisito 1.2.1.2.

Para la comprobación del requisito 1.2.1.1, se realiza la siguiente prueba: se hace uso, por ejemplo, del botón de avance del modo manual de la aplicación, como se muestra en la Figura 4.8, de tal forma, que el comando asignado a dicha acción debe ser recibido en el microcontrolador ESP32 y TIVA, dicho comando corresponde con el número '4'. Para comprobar el flujo de dicho comando, se hace uso de la herramienta cliente MQTTX para corroborar que el comando se publica en el cliente, el terminal correspondiente al microcontrolador ESP32 para confirmar que ha recibido el comando del cliente MQTT, y, por último, el terminal correspondiente al microcontrolador TIVA para verificar que el ESP32 ha transmitido correctamente el comando por el módulo UART y realiza la acción conveniente.

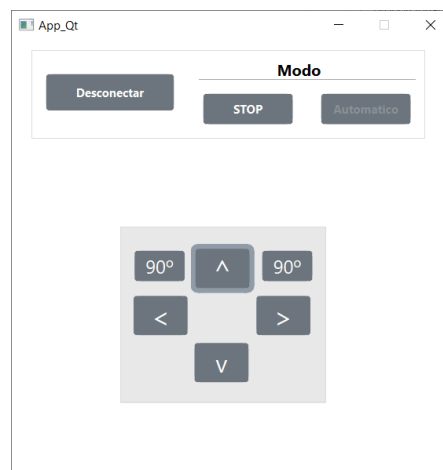


Figura 4.8. Acción de avance para prueba del requisito 1.2.1.1

A continuación, se muestra, en la Figura 4.9, la recepción del comando por parte del cliente MQTT.

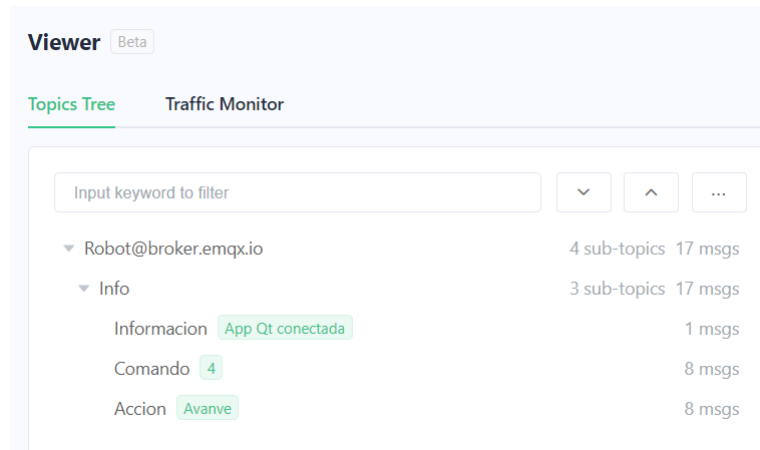


Figura 4.9. Recepción del comando en el cliente MQTT para la prueba del requisito 1.2.1.1

Por último, expone, en la Figura 4.10, la recepción del comando por parte del microcontrolador ESP32 y TIVA.

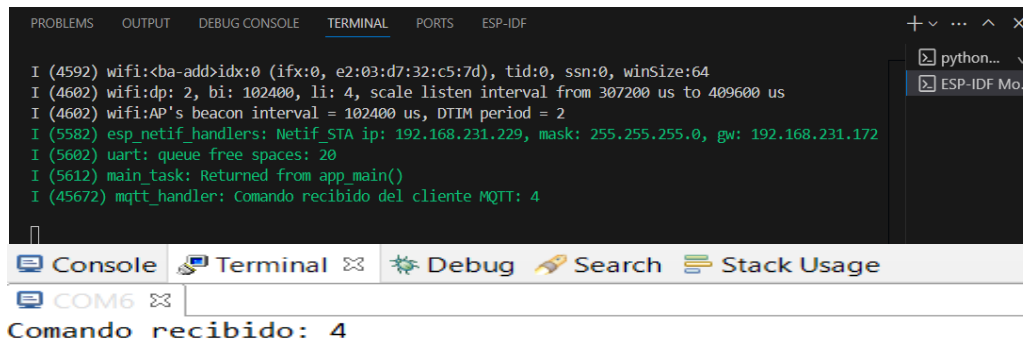


Figura 4.10. Recepción del comando en el ESP32 y TIVA

Tras la realización de dicha prueba, se comprueba el satisfactorio cumplimiento del requisito 1.2.1.1, y, con ello, también se confirma el correcto flujo de la información desde la aplicación Qt hasta el microcontrolador TIVA.

Para la comprobación del requisito 1.2.1.2, se realiza la prueba efectuada en la confirmación del requisito 1.1.3, pero ahora la aplicación debe obtener la información del cliente MQTT y dibujarla en el gráfico destinado a ello. Por tanto, se envía el mensaje desde el microbot, el cual es “0.1.1.1.0.1.1”, el mensaje varía para una mejor representación, que corresponde con: el microbot se encuentra en las coordenadas (0,1), tiene paredes a la derecha e izquierda, y un movimiento con sentido positivo y vertical.

A continuación, en la Figura 4.11, se muestra el resultado de la recepción del mensaje por parte de la aplicación y su representación en el gráfico.

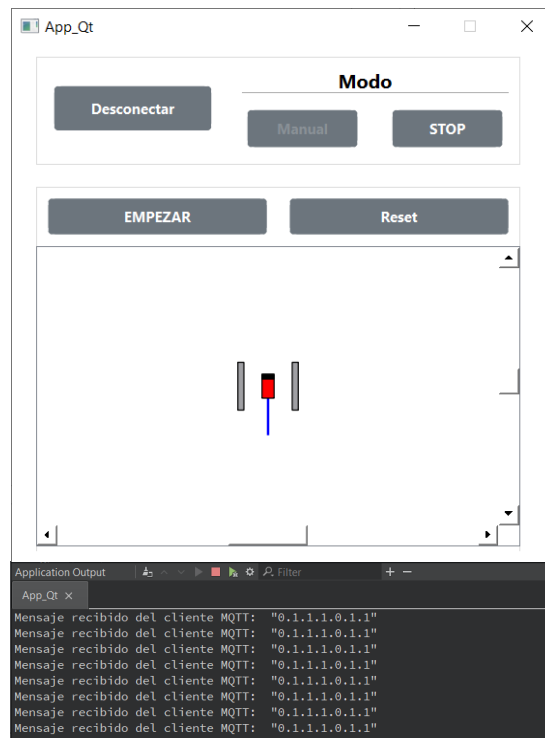


Figura 4.11. Prueba del requisito 1.2.1.2

Tras la prueba, se observa que la aplicación monitoriza correctamente la información enviada por el microbot, y, en consecuencia, el cumplimiento del requisito 1.2.1.2.

Con la verificación de este último requisito, se puede afirmar la satisfacción del requisito 1.2.1, que engloba los requisitos vistos anteriormente, y, por tanto, del requisito 1.2, referente a la aplicación Qt. También, se demuestra el correcto flujo de la información en ambos sentidos, es decir, tanto con origen en el microcontrolador TIVA y destino en la aplicación, como con origen en la aplicación y destino en el microcontrolador TIVA.

Para finalizar el capítulo, se añade la Tabla 4.1, donde recogen las pruebas realizadas para el requisito en cuestión, así como el resultado de éstas.

Nº	Nombre o ID del requisito	Descripción de la prueba	Resultado
1	1.1.1.1	Prueba de las funciones que controlan los motores, y, por tanto, el movimiento del microbot.	Satisfactorio
2	1.1.1.2	Prueba de las funciones que realizan los giros del microbot.	Satisfactorio
3	1.1.2	Prueba de las funciones que leen los sensores de distancia.	Satisfactorio
4	1.1.3.1	Prueba de comunicación entre TIVA y ESP32, mediante el módulo UART de ambos.	Satisfactorio
5	1.1.3.2	Prueba de comunicación entre ESP32 y el cliente MQTT, mediante el módulo wifi.	Satisfactorio
6	1.1.4	Prueba para comprobar el funcionamiento del sensor bajo del microbot.	Satisfactorio
7	1.2.1.1	Prueba de control del microbot desde la aplicación Qt.	Satisfactorio
8	1.2.1.2	Prueba de monitorización del movimiento del microbot en la aplicación Qt.	Satisfactorio

Tabla 4.1. Tabla de las pruebas realizadas

Capítulo 5. Conclusiones y trabajo futuro

Este capítulo final se divide en dos partes, por un lado, se habla del posible trabajo futuro que proporcione una mejora, optimización e innovación del presente proyecto, y, por otro lado, se comentan las conclusiones obtenidas del trabajo realizado.

5.1 Trabajo futuro

Con la visión futura de perfeccionar y dar mayor versatilidad al proyecto realizado, dentro del campo didáctico en el cual se encuentra, surgen nuevas ideas y conceptos que pueden ayudar al logro de ello.

Posibles trayectos para conseguirlo pueden ser los siguiente:

- Corrección de la trayectoria. Ya que uno de los principales problemas encontrados en el proyecto, referente al microbot, es la desviación de su trayectoria por parte de los motores, se puede realizar una rutina o tarea, que se encargue de ello, haciendo uso de los componentes con los que ya cuenta el microbot, podría obtenerse por dos vías:

Bien, mediante el sensor brújula, ya que la funcionalidad dada a este sensor es la de obtener el ángulo respecto del norte magnético, se podría implementar una rutina que cada un breve periodo de tiempo compruebe este ángulo para así comprobar si la trayectoria se está desviando y en caso afirmativo corregirla en la dirección oportuna. Pero esta vía conlleva

otro problema encontrado, que es el ruido introducido en la línea de tierra por parte de los motores cuando estos están en funcionamiento, el cual produce grandes variaciones en las medidas del sensor brújula, por lo que habría que diseñar otra línea de tierra independiente o hacer uso de condensadores de desacoplo.

O bien, mediante los sensores de distancia laterales, de igual forma se podría implementar una tarea que compruebe la distancia a la que se encuentran las paredes del laberinto y así comprobar si existe desviación. Con esta opción, desde el punto de vista del ejercicio, habría que tener en cuenta que en ciertos momentos no se detectaran paredes, ya que será la boca a una nueva calle del laberinto.

- Representación de la trayectoria y el entorno en cada momento. Puesto que el microbot obtiene la información de su entorno cada cierta distancia recorrida, se puede mejorar dicho mecanismo para que analice el entorno y envíe la información constantemente para su representación. En dicho mecanismo habría que tener en cuenta que se trata de una comunicación inalámbrica y pueden producirse retardos, y con ello la pérdida de información.
- Mayor funcionalidad al modo manual. Ya que el modo manual únicamente hace uso de los motores, una innovación en este modo podría ser la incorporación de una pequeña cámara que muestre en la aplicación Qt el entorno del microbot y así poder controlarlo sin necesidad de tenerlo enfrente.
- Aportación de más información en la aplicación Qt. Partiendo de los sensores usados, podría incluirse la velocidad que tiene el microbot en cada momento, usando los sensores encoders, de igual forma se podría añadir la temperatura del entorno, gracias al sensor brújula.

Por último, a modo dar mayor visibilidad y utilidad, en proyectos futuros, a la herramienta Tiva C Series PinMux, proporcionada por el fabricante Texas Instruments [21], véase la Figura 6.1, la cual permite seleccionar los módulos que serán usados en el proyecto, así como configurar los pines deseados como entradas o salidas. La aplicación permite generar archivos `.c` y `.h` con las configuraciones pertinentes, que deberán ser completadas por el usuario en

A nivel de software de alto nivel, se ha adquirido experiencia en el desarrollo de interfaces gráficas con Qt y el uso de CMake como sistema de construcción. Asimismo, se ha aprendido a estructurar correctamente un proyecto modular, integrar drivers propios, y emplear herramientas de depuración tanto en entornos embebidos como en el desarrollo de aplicaciones de escritorio.

Durante el proceso de desarrollo se han detectado tanto fortalezas como limitaciones en las tecnologías empleadas. El uso del microcontrolador TIVA ha resultado adecuado para controlar periféricos y gestionar tareas concurrentes mediante FreeRTOS, aunque se han evidenciado ciertas restricciones en la gestión de memoria y conectividad frente a plataformas más modernas como ESP32.

En cuanto al uso de MQTT como protocolo de comunicación, ha demostrado ser una opción ligera y efectiva para la comunicación remota entre la aplicación Qt y el microbot. Sin embargo, su integración con el entorno embebido implicó una curva de configuración adicional y la necesidad de comprender detalladamente la pila de red.

Una de las principales dificultades técnicas encontradas ha sido la inestabilidad generada por sensores y actuadores de bajo coste. Éstos mostraban lecturas ruidosas o con alta variabilidad, lo que dificultaba la toma de decisiones precisas por parte del robot y reducía la fiabilidad de los movimientos automáticos. Esto obligó a implementar filtrados adicionales y validaciones, aumentando la complejidad del sistema y afectando al rendimiento general.

Desde un punto de vista metodológico, el diseño por bloques y el uso de diagramas UML han facilitado la organización y documentación del sistema. No obstante, una planificación más detallada en las fases iniciales habría permitido prever con mayor antelación ciertos problemas de integración entre hardware y software.

Apéndice A. Manual de instalación y uso

El presente capítulo recoge las guías necesarias para la programación de los microcontroladores empleados en este proyecto, así como un manual de uso centrado en la aplicación Qt. Este proyecto se ha desarrollado haciendo uso del sistema operativo Windows, por lo que, los entornos de desarrollo han sido instalado para tal sistema operativo, pero éstos cuentan también con soporte para Linux y macOS.

A.1 Manual de instalación

Este apartado expone una guía del proceso necesario para añadir el software correspondiente a cada núcleo que constituye dicho proyecto. Por tanto, dicho manual se divide en tres apartados.

A.1.1 Programación del microcontrolador TIVA

Para realizar este proceso, es necesario contar con el entorno de desarrollo, Code Composer Studio, el cual es proporcionado por el fabricante Texas Instruments [15]. Además, se debe obtener el proyecto correspondiente a este microcontrolador a través del enlace a GitHub proporcionado.

Una vez se haya realizado lo comentado anteriormente, solo es necesario añadir el proyecto al entorno de desarrollo y cargarlo en el microcontrolador. Para ello, se deben seguir los siguientes pasos:

1. Iniciar el entorno de desarrollo y seleccionar el directorio que se usará como espacio de trabajo, en la Figura A.1 se puede observar un ejemplo.

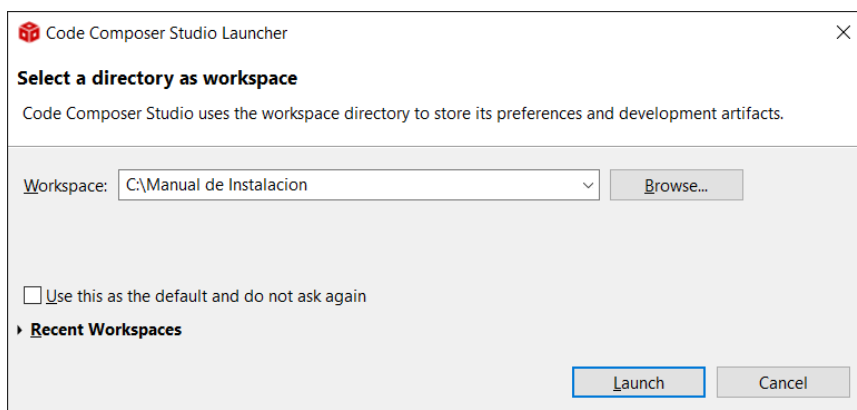


Figura A.1. Directorio utilizado como espacio de trabajo.

2. Añadir el proyecto al espacio de trabajo. Se puede realizar de varias formas desde el menú superior, siguiendo las siguientes indicaciones:

“File” → “Import” → “Code Composer Studio” → “CCS Projects”.

O bien:

“Project” → “Import CCS Projects”.

Seguidas estas indicaciones, aparecerá la ventana mostrada en la Figura A.2, donde seleccionaremos el proyecto

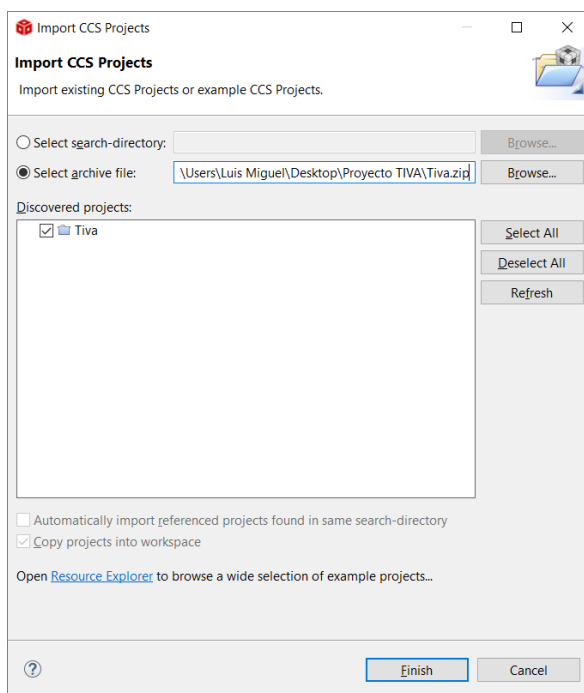


Figura A.2. Selección del directorio del proyecto

Tras seleccionar el proyecto, pulsar en *“Finish”* y el proyecto ya estará incluido en el entorno de desarrollo.

3. Cargar el proyecto en el microcontrolador. Para ello, hay que conectar el microcontrolador al PC, mediante su puerto *Debug* y seguir las siguientes indicaciones:

“Run” → “Debug”.

Otras opciones son: presionando la tecla F11 o haciendo uso del icono mostrado en la Figura A.3.

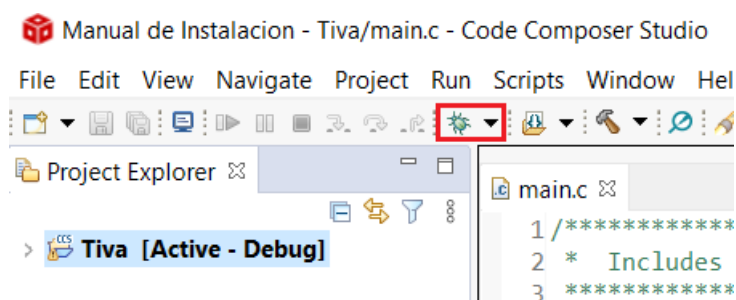


Figura A.3. Icono para cargar el proyecto

Una vez se haya completado la carga del proyecto, el microcontrolador TIVA estará listo para su uso.

A.1.2 Programación del microcontrolador ESP32

De igual forma que en el apartado anterior, es necesario descargar el proyecto correspondiente a este microcontrolador a través del enlace de GitHub. Además, se debe hacer uso del entorno de desarrollo, Visual Studio Code [17], otra opción es usar el entorno de desarrollo propio del fabricante del microcontrolador, pero no es el caso de este proyecto.

Para añadir el proyecto, basta con incluirlo en forma descomprimida, como se muestra en la Figura A.4 o bien mediante las siguientes indicaciones:

“File” → “Add Folder to Workspace”

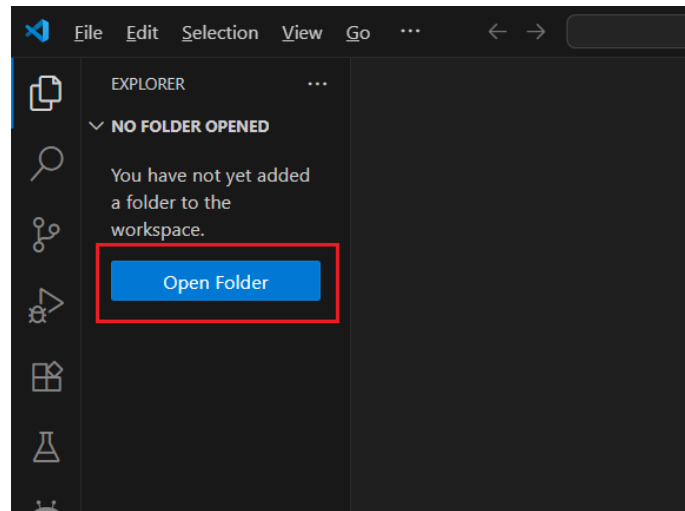


Figura A.4. Añadir proyecto en Visual Studio Code

Una vez añadido, se puede guardar el espacio de trabajo, si se quiere, mediante las indicaciones:

“File” → “Save Workspace Ass...”

Para cargar el proyecto en el microcontrolador, se hace uso del icono que se encuentra en el menú inferior del entorno de desarrollo, mostrado en la Figura A.5.

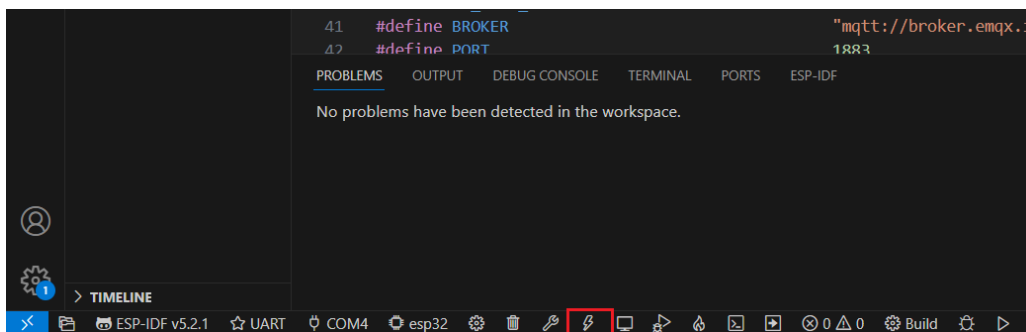


Figura A.5. Cargar proyecto en Visual Studio Code

Finalizado el proceso de carga del proyecto, el microcontrolador ESP32 se encuentra preparado para su uso.

A.1.3 Instalación de la aplicación Qt

Para el uso y puesta en marcha de la aplicación no es necesario el entorno de desarrollo pertinente, ya que en el enlace GitHub se proporciona una versión *Release* de la aplicación lista para su uso. Solo es necesario ejecutar el archivo .exe que se proporciona en la carpeta del proyecto.

A.2 Manual de uso

El uso del sistema desarrollo se basa en la aplicación Qt implementada, por tanto, el presente manual de uso consiste en una guía sobre el manejo de dicha aplicación.

En primer lugar, es necesario establecer la conexión entre el microbot y la aplicación, haciéndose ésta a través del cliente MQTT incorporado en ambas partes. Para ello, se hace uso del botón indicado en la Figura A.6, encargado de la conexión/desconexión con dicho cliente.

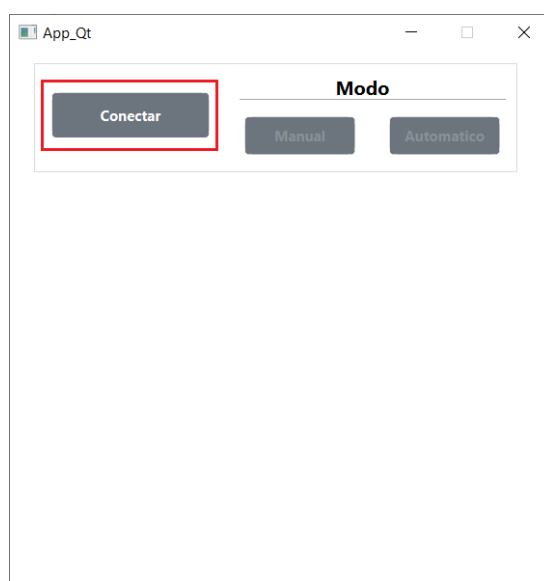


Figura A.6. Botón para conexión/desconexión

Una vez realiza esta acción, la aplicación se encuentra conectada al cliente, y en consecuencia el botón indicado cambia su funcionalidad para permitir la acción opuesta.

Realizada la conexión, se habilitan las opciones de funcionamiento, indicadas en la Figura A.7, permitiendo la selección entre el modo manual y automático del microbot. Únicamente un modo puede ser seleccionado al mismo tiempo, por tanto, el modo no seleccionado se deshabilita.

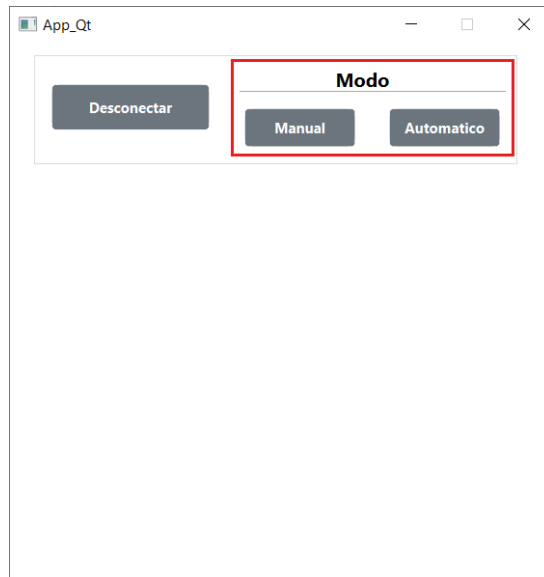


Figura A.7. Selección del modo de funcionamiento

En caso de seleccionar el modo manual, se permite el acceso a la botonera encargada del manejo del movimiento del microbot, como se aprecia en la Figura A.8. Mediante dicha botonera, se puede hacer avanzar y retroceder al microbot, así como la realización de giros momentáneos o de 90 grados en ambas direcciones. Para abandonar dicho modo de funcionamiento, se utiliza el mismo botón usado para la selección de éste.

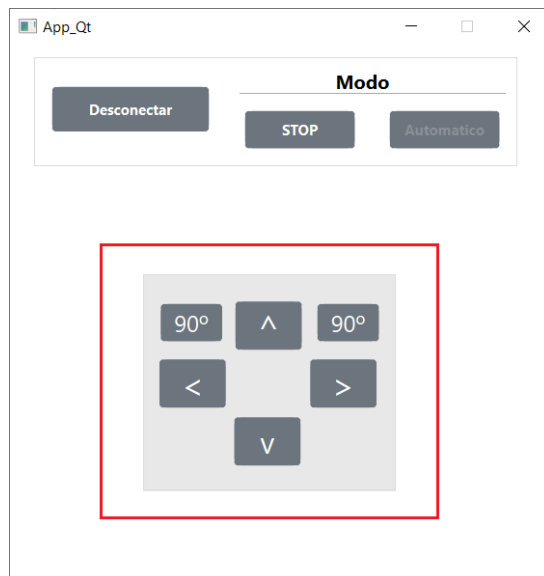


Figura A.8. Botonera para manejar el microbot en modo manual

En caso de seleccionar el modo automático, se tiene acceso tanto a los botones que manejan la navegación del microbot, como del gráfico donde se representa el movimiento y entorno del microbot durante la navegación, como se aprecia en la Figura A.9.

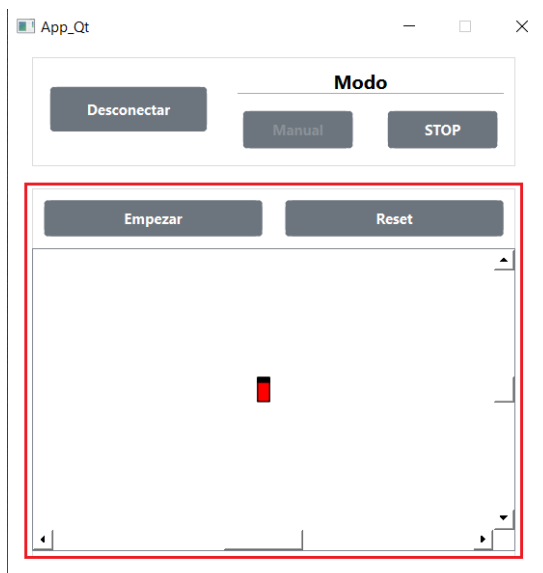


Figura A.9. Botones y gráfico del modo automático

Para dar inicio al algoritmo de navegación del microbot, se hace uso del botón, indicado en la Figura A.10, además, este botón permite realizar una pausa del algoritmo de navegación y poder reanudarlo en el instante donde se detuvo.

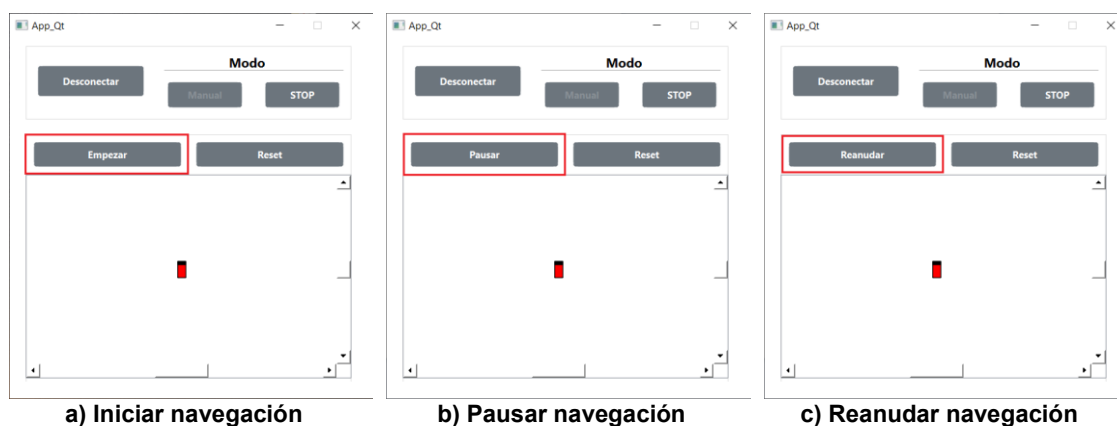


Figura A.10. Botón para iniciar, pausar y reanudar la navegación

En el caso de que se desee hacer un reinicio del algoritmo de navegación, se utiliza el botón indicado en la Figura A.11, mediante esta acción se limpia el gráfico que representa el movimiento y entorno del microbot, y se restaura el estado del botón que da inicio a la navegación, permitiendo la realización de nuevos intentos.

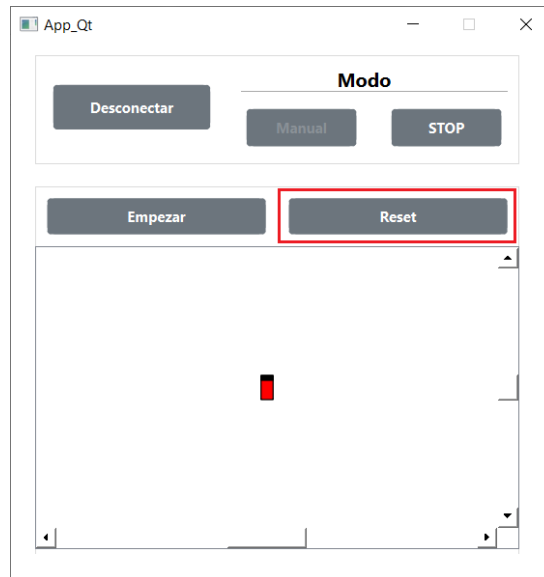


Figura A.11. Reinicio del algoritmo de navegación

Por último, indicar que, en caso de abandonar el modo automático, mediante el mismo botón utilizado para su selección, mientras se esté realizando la navegación del microbot, esto supondrá un reinicio del algoritmo de navegación igual al uso del botón de reinicio comentado anteriormente.

Apéndice B. Presupuesto de elaboración

La siguiente Tabla B.1, muestra los costes actuales de los componentes y materiales fungibles utilizados en la elaboración del proyecto.

Componente	Coste unitario (€)	Unidades empleadas	Coste total (€)
Soldador de estaño	11,99€	1N	11,99€
Estaño	5,99€	1N	5,99€
Flux	11,94€	1N	11,94€
Pines y placas pretaladradas	10,99€	1N	10,99€
Cables pines	9,99€	1N	9,99€
Motores	10,69€	2N	21,38€
Sensor Distancia	6,4€	3N	19,2€
Sensor Brújula	6,99€	1N	6,99€
Sensor Encoder	0,85€	1N	0,85€
Sensor Suelo	0,85€	1N	0,85€
Tiva	36,3€	1N	36,3€
ESP32	7,99€	1N	7,99€
Batería	22,79€	1N	22,79€
Material para prototipo	100€	1N	100€
		Total	267,25€

Tabla B.1. Coste de componentes y material fungible

En el cálculo expuesto en la Tabla B.2, el precio unitario del ordenador ya introduce los costes de las licencias usadas, correspondientes al sistema operativo y software empleado para la redacción.

Equipo/software	Coste unitario (€)	Periodo de amortización	Periodo de uso	Coste total (€)
Ordenador	1100€	36 meses	7 meses	213,89€
Total				213,89€

Tabla B.2. Amortización de equipos y licencias de software

Para el cálculo del coste por hora, se hace uso del rango de salarios publicado por el COIT, Colegio Oficial de Ingenieros de Telecomunicación, el cual corresponde de 20000€ a 30000€, pero para tener un valor más exacto, se realiza el cálculo basándose en fuente similares al COIT [22], donde el salario mínimo de un ingeniero junior es de 20600€ brutos al año.

Por tanto, partiendo de un salario anual bruto de 20600€, una jornada laboral típica de España de 40 horas a la semana y unas 52 semanas laborales anuales, se obtienen 2080 horas laborales al año. Descontando vacaciones y festivos, que rondan aproximadamente 220 horas, se tienen en torno a 1860 horas efectivas anuales. Por lo que de la Ecuación B.1, se obtiene el coste por hora:

$$\text{Coste por hora} = \frac{\text{Salario bruto anual}}{\text{Horas laborales anuales}} = \frac{20600\text{€}}{1860 \text{ horas}} \approx 11,08 \text{ €/h} \quad (\text{B.1})$$

Una vez calculado el coste por hora, se determina en la Tabla B.3 el coste total de la mano de obra empleada en la realización del proyecto.

Actividad	Horas
Coordinación del TFG	15
Análisis de requisitos	10
Desarrollo hardware/software del prototipo	210
Realización de las pruebas	8
Redacción de la memoria	60
Preparación de la presentación	12
Horas totales	315
Coste por hora	11,08€
Coste total de la mano de obra	3490,2€

Tabla B.3. Horas invertidas en la realización del proyecto (mano de obra)

Por último, se obtiene el coste total del proyecto, recogido en la Tabla B.4.

Concepto	Coste (€)
Coste de componentes y material fungible	267,25€
Amortización de equipos y licencias de software	213,89€
Mano de obra	3490,2€
Coste total	3971,34€

Tabla B.4. Presupuesto total

Referencias

- [1] EmpresaActual, «EmpresaActual,» 2 Febrero 2022. [En línea]. Available: <https://www.empresaactual.com/evolucion-y-tendencias-de-la-robotica-industrial/>. [Último acceso: 24 Agosto 2023].
- [2] A. O. Baturone, Robótica: manipuladores y robots móviles, Marcombo, 2005.
- [3] R. Foran, Robotics: from automatons to the roomba, Minneapolis, Minnesota: ABDO, 2015.
- [4] Edsrobotics, «Edsrobotics,» 2 Febrero 2021. [En línea]. Available: <https://www.edsrobotics.com/blog/evolucion-robotica-industrial/>. [Último acceso: 24 Agosto 2023].
- [5] Robotsinaction, «Robot in Action,» 7 Febrero 2019. [En línea]. Available: <https://robotsinaction.com/el-primer-robot-industrial-unimate/>. [Último acceso: 24 Agosto 2023].
- [6] Krejčí, Jakub and Babiuch, Marek and Suder, Jiří and Krys, Václav and Bobovský, Zdenko, «Internet of Robotic Things: Current Technologies, Challenges, Applications, and Future Research Topics,» *Sensors*, vol. 25, nº 3, p. 765, 2025.

- [7] T. Instruments, «TI.com,» Texas Instruments Incorporated, [En línea]. Available: <https://www.ti.com/product/es-mx/TM4C123GH6PM#params>.
- [8] Ardunaut, «arduining.com,» Arduining, 17 Abril 2015. [En línea]. Available: <https://arduining.com/2015/04/17/digital-to-analog-with-tiva-c-launchpad-and-energia/>.
- [9] E. Systems, «espressif.com,» [En línea]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf.
- [10] Circuits4you, «circuits4you.com,» Circuits4you, 31 Diciembre 2018. [En línea]. Available: <https://circuits4you.com/2018/12/31/esp32-wroom32-devkit-analog-read-example/>.
- [11] S. CORPORATION, «SHARP,» [En línea]. Available: <https://global.sharp/>.
- [12] I. Vishay Intertechnology, «VISHAY,» [En línea]. Available: <https://www.vishay.com/>.
- [13] AZ-Delivery, «Az-Delivery,» [En línea]. Available: <https://www.az-delivery.de/es>.
- [14] F. Corporation, «Futaba,» [En línea]. Available: <https://www.futaba.co.jp/en>.
- [15] T. Instruments, «TI.com,» Texas Instruments Incorporated, [En línea]. Available: <https://www.ti.com/tool/CCSTUDIO>.
- [16] R. T. E. Ltd., «FreeRTOS - Market leading RTOS (Real Time Operating System) for embedded systems,» FreeRTOS.org, [En línea]. Available: <https://www.freertos.org/>.
- [17] Microsoft, «Visual Studio Code,» [En línea]. Available: <https://code.visualstudio.com/>.
- [18] E. Systems, «Examples — ESP-IDF Programming Guide,» GitHub, [En línea]. Available: <https://github.com/espressif/esp-idf/tree/v5.4.2/examples>.

- [19] Q. Group, «Download Qt,» Qt Official Website, [En línea]. Available: <https://www.qt.io/download>.
- [20] E. T. Inc., «MQTTX,» MQTTX Download, [En línea]. Available: <https://mqttx.app/downloads>.
- [21] T. Instruments, «Texas Instruments,» Texas Instruments Incorporated, [En línea]. Available: https://www.ti.com/tool/TM4C_PINMUX.
- [22] «Jobted,» [En línea]. Available: <https://www.jobted.es/salario/ingeniero-telecomunicaciones?>.
- [23] V. I. Incorporation, «VISHAY,» [En línea]. Available: <https://www.vishay.com/docs/83752/tcrt1000.pdf>.
- [24] AZ-Delivery, «Az-Delivery,» [En línea]. Available: https://cdn.shopify.com/s/files/1/1509/1638/files/GY-_271_Triple_Axis_Compass_Datenblatt.pdf?72630.
- [25] S. CORPORATION, «SHARP,» [En línea]. Available: https://global.sharp/products/device/lineup/data/pdf/datasheet/gp2y0a41sk_e.pdf.
- [26] I. Vishay Intertechnology, «VISHAY,» [En línea]. Available: <https://www.vishay.com/docs/83751/cny70.pdf>.
- [27] T. I. Incorporated, «Texas Instruments,» [En línea]. Available: <https://tinyurl.com/TM4C123GH6PM>.
- [28] F. CORPORATION, «Futabausa,» [En línea]. Available: <https://futabausa.com/wp-content/uploads/2019/07/S3003.pdf>.